

It Is Not the Signature: A Pre-Authentication Key-Parse Attempt Dominates JSON Web Token Validation Cost and Is Coupled to a Key-Confusion Defence

Jannatul Ferdous Katha^{a,c}, Tasmia Tahmid Prova^{b,c}, Md Kishor Morol^c, Tze Hui Liew^{d,*}, Dip Nandi^{c,e}

^a*Department of CSE, Bangladesh University of Professionals (BUP), Dhaka, Bangladesh*

^b*Department of CSE, Bangladesh University of Business and Technology (BUBT), Dhaka, Bangladesh*

^c*ELITE Research Lab, Queens, NY, USA*

^d*Faculty of Information Science and Technology, Multimedia University, Melaka, Malaysia*

^e*Department of Computer Science, American International University-Bangladesh (AIUB), Dhaka, Bangladesh*

Abstract

JSON Web Token (JWT) validation is the authentication step of most micro-service requests, and its per-request cost is routinely attributed to cryptography. In `jsonwebtoken`, a widely used Node.js implementation, the signature primitive is 2.208 μ s of a 29.92 μ s validation. The dominant component, 18.04 μ s (60%), is a discarded key-parse attempt: a shared secret supplied as a string is first offered to the asymmetric key parser and is resolved only after that parse fails. The attempt runs before the algorithm check and before signature verification, so any well-formed token, forged or not, triggers it. It entered the library with the security release that fixed an RSA-to-HMAC key-confusion vulnerability, and it is load-bearing: removing it the obvious way fails the library's own key-confusion tests. Its price is set, causally, by the bundled OpenSSL release: the same calls issued from C cost 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8. In a deployed service, validation adds 384.6 μ s of processor time per request, within 0.8% of the microbenchmark

*Corresponding author.

Email addresses: `ferdous.katha@elitelab.ai` (Jannatul Ferdous Katha), `tasmia.prova@elitelab.ai` (Tasmia Tahmid Prova), `kishormorol@ieee.org` (Md Kishor Morol), `thliew@mmu.edu.my` (Tze Hui Liew), `dip.nandi@aiub.edu` (Dip Nandi)

prediction. Of seven JWT libraries in four languages, only this one attempts the parse. We give a narrow fix that preserves the key-type defence and makes validation 4.15 times faster, show that a client controlling the token header can still reach the parse under it, and identify pre-parsed keys as the remedy that removes it for every input. In our sample, 94% of call sites pin no algorithm and none pre-parses the key. The defect has been reported upstream.

Keywords: JSON Web Token, authentication, key confusion, cryptographic API, OpenSSL, Node.js, performance measurement

1. Introduction

Authentication in cloud-native services is commonly described as imposing a per-request cryptographic cost (Zhu et al., 2023). The assumption is old and has been tested before, one layer down: Coarfa et al. (2006) instrumented TLS web servers and found RSA responsible for only a minority of server cost, the remainder falling to symmetric operations, hashing and protocol handling. Our result is that argument repeated at the application layer, and reaching a sharper version of the same conclusion.

The reasoning being tested is intuitive: validating a JSON Web Token (Jones et al., 2015) means verifying a signature, signatures are cryptography, and cryptography is expensive relative to parsing. The conclusion usually drawn is that the cost should be *moved* — to a sidecar proxy, a gateway, or dedicated hardware — because specialised components do cryptographic work well.

We report that the premise is wrong for the most widely used JavaScript implementation we could measure. The natural repair is wrong as well. Once the signature primitive is shown to be a small share of the cost, the remainder is readily attributed to key material being converted from a string into a key object on every call. Measuring that conversion directly puts it at 0.583 μ s: too small to be the explanation for anything.

The actual mechanism is a control-flow accident, shown in Figure 1. Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed. Every validation therefore constructs and discards an OpenSSL parse failure, at 18.04 μ s against a total call cost of 29.92 μ s.

27 We are careful about what that 18.04 μ s is. It is the cost of the whole
28 discarded attempt: the work OpenSSL performs before deciding the input is
29 not an asymmetric key, plus the construction and unwinding of the exception
30 that carries that decision back to the library. Section 4.2 separates the two,
31 by reissuing the OpenSSL calls from C and by measuring an exception of
32 the same shape with no OpenSSL call. Where the attempt is expensive it is
33 almost all parse; the exception is a few microseconds. The title’s “key-parse
34 attempt” is therefore the accurate name, and “discarded exception” would
35 not be.

36 How much the attempt costs depends on which OpenSSL release performs
37 it, by more than an order of magnitude (Section 5). Runtimes carrying an
38 OpenSSL release from 3.2 onward, which include the current Node.js release
39 lines, pay tens of microseconds per validation; runtimes carrying the 3.0
40 series pay hundreds. The 3.0 series is not confined to retired Node.js releases.
41 It is the system OpenSSL of long-term-support distributions still in wide
42 use, among them Ubuntu 22.04 and Debian 12, and a Node.js build that
43 links the system library rather than its bundled copy inherits it; so does any
44 service whose image is pinned to an older base layer, which is common in
45 practice (Zerouali et al., 2019). At either price the discarded attempt is the
46 largest single component of the call, and in every case it is avoidable.

47 The cost is also a security matter, in three ways (Section 8). First, it is
48 paid before authentication: key resolution precedes both the algorithm check
49 and the signature check, so every request carrying a well-formed token pays it,
50 whether or not anyone the service trusts issued that token. Second, it did not
51 arrive through neglect. The resolution block entered the library in version 9.0.0,
52 the release that fixed an RSA-to-HMAC key-confusion vulnerability (Auth0,
53 2022), and the key-type checks that implement that fix read the type of the key
54 object the block produces. Third, it therefore cannot be removed the obvious
55 way: dispatching on the token’s declared algorithm without restricting which
56 key material may take the fast path fails the library’s own key-confusion tests.
57 A performance change to authentication code is a change to its security, and
58 we analyse it as one.

59 The result is broader than one library in two respects. The design that
60 produces the cost — resolving key material on every call, by way of a
61 parse whose failure serves as a type test — can be named independently of
62 `jsonwebtoken`, and Section 7 shows it predicts which of seven libraries pay.
63 And the reason the cost went unattributed is methodological: the component
64 that sets its price, the bundled OpenSSL release, is one that measurement

65 protocols rarely record at the granularity at which it changes.

66 *Research questions.*

67 **RQ1** Where does the per-request cost of token validation actually go?

68 **RQ2** What governs the magnitude of that cost?

69 **RQ3** Does it matter in a deployed service, and how common is the pattern
70 that avoids it?

71 **RQ4** Is the defect a property of one library, or of a design choice other
72 libraries could share?

73 **RQ5** How does the cost relate to the library’s security: who can cause it, why
74 is it there, and can it be removed without weakening the key-confusion
75 defence?

76 *Contributions.* All supported by measurements in the replication pack-
77 age (Katha et al., 2026).

- 78 1. A decomposition of the validation call, by two independent methods,
79 locating its dominant component in a discarded key-parse attempt
80 rather than in the signature primitive or in key conversion, and a
81 second decomposition of that attempt into the OpenSSL parse, the V8
82 exception and the binding between them (Section 4).
- 83 2. A security analysis of that component (Section 8). It is reachable before
84 authentication by any well-formed token; it entered the library with a
85 key-confusion fix and is load-bearing for it, so the natural optimisation
86 fails the library’s key-confusion tests; a narrow fix preserves the defence
87 and makes validation 4.15 times faster, but leaves the parse reachable
88 through an attacker-chosen algorithm header; and only a pre-parsed
89 key removes it for every input.
- 90 3. A causal attribution of its magnitude to the bundled OpenSSL release:
91 the parse reissued from C with no JavaScript engine across 8 identically
92 built OpenSSL releases, a runtime pair that crosses engine and library,
93 and a repetition on a second architecture (Section 5).
- 94 4. A deployed-service measurement across seven arrival rates, and a survey
95 establishing that the avoiding pattern is rare in three samples of public
96 code (Section 6).
- 97 5. A comparison of seven JWT libraries in four languages showing that
98 the cost follows a design choice — resolving key material on every call,
99 of which a failing asymmetric parse used as a type test is the costliest
100 form — rather than JWT validation as such (Section 7).

101 2. Background and related work

102 *What a validation call does..* A JSON Web Token carries a header, a payload
103 and a signature over the first two (Jones et al., 2015). Validating one means
104 decoding two segments, resolving key material into a form the cryptographic
105 library accepts, and verifying the signature. The middle step — key resolution
106 — is the subject of this paper, and it is the step most often left implicit when
107 validation cost is discussed.

108 *Where the cost is usually attributed..* Published measurement of layered
109 enforcement in microservices concentrates on transport and proxy layers.
110 Zhu et al. (2023) decompose service-mesh sidecar overhead and show that
111 the dominant contributors depend on configuration and workload rather
112 than being a fixed tax; we reach an analogous conclusion one layer up, for
113 application-level validation. Application-level token validation is itself thinly
114 evidenced, and is commonly reported as a difference between scenario averages,
115 which attributes to validation every difference between two runs. We were
116 unable to locate a study that measures the key-resolution step directly; we
117 state that as an absence we searched for rather than as a claim about the
118 whole literature.

119 *Cryptographic cost has been over-attributed before..* Coarfa et al. (2006) re-
120 mains the clearest precedent: measuring TLS web servers rather than primi-
121 tives, they found the public-key operation was a minority of server cost. Work
122 on cryptographic library performance since has largely stayed at the level of
123 the primitive or the protocol exchange (Naser et al., 2020; Sikeridis et al.,
124 2020), with Tuveri and Brumley (2019) a notable exception in examining how
125 a library loads and dispatches implementations rather than how fast those
126 implementations run. Our result concerns neither: it is the cost of getting
127 key material *into* the library.

128 *Performance bugs as a class..* Jin et al. (2012) studied real-world performance
129 bugs across five code bases and found that most follow a small number of
130 recurring patterns — among them uncoordinated and skippable function calls
131 — and that fixes are usually local. Zaman et al. (2012) found, from 400 Firefox
132 and Chrome reports, that performance bugs differ from other bugs in how
133 they are reported, discussed and fixed, in part because reproducing them is
134 hard; Nistor et al. (2013a) report that most are found by code reasoning rather
135 than by observing symptoms. Liu et al. (2014) reach similar conclusions for

136 mobile applications. Detection work targets structural symptoms: redundant
137 traversals with similar memory-access patterns (Nistor et al., 2013b), or
138 predicates that separate slow from fast runs (Song and Lu, 2014). The defect
139 here is of the “skippable call” kind Jin et al. describe, but it has none of the
140 structural signatures those detectors look for: it is one call, executed once
141 per request, whose failure is its only observable effect. It is also the shape
142 that makes a performance bug hard to report: our own upstream report was
143 preceded by a two-year-old report of the symptom with no diagnosed cause
144 (Section 8.6).

145 *JavaScript and Node.js performance..* Selakovic and Pradel (2016) classified
146 performance issues in JavaScript projects and found that inefficient API usage
147 — correct calls made in a costly way — is among the most common root causes,
148 and that the fixes are typically a few lines. Their later profiler (Selakovic et al.,
149 2017) targets precisely the *order* in which a program evaluates conditions, the
150 axis on which this defect lives: a check that almost always fails is evaluated
151 before one that almost always succeeds. Gong et al. (2015) locate code that
152 defeats JIT optimisation; exception-heavy control flow is one such pattern,
153 and Section 4.2 measures what it costs here. Work on the Node.js event loop
154 shows why a per-request synchronous cost matters beyond its magnitude: a
155 single slow handler delays every other client (Davis et al., 2018; Staicu and
156 Pradel, 2018). We make no availability or capacity claim, but the mechanism
157 by which a synchronous per-request cost propagates is the same.

158 *Version and environment variation..* That software performance changes
159 across versions in ways maintainers do not anticipate is established: Kalte-
160 necker et al. (2023) track performance across configurable systems’ release
161 histories, and Chen and Shang (2017) study the code changes that introduce
162 regressions. Laaber and Leitner (2018) assess whether microbenchmark suites
163 detect such changes at all, and Stefan et al. (2017) find performance unit
164 testing rare in practice. Container images make this concrete by pinning a
165 dependency set implicitly: Cito et al. (2017) characterise how the ecosystem
166 uses them, Zerouali et al. (2019) show that images routinely ship outdated
167 packages, and Malka et al. (2026) show that an image tag does not guarantee
168 a reproducible build. For OpenSSL 3 specifically, the maintainers of a major
169 downstream library have stated publicly that parsing and key loading re-
170 gressed substantially against 1.1.1 (Gaynor and Kehrer, 2026); that statement
171 is not peer reviewed and was not produced under a protocol we can assess,

172 and Section 5.5 supplies a controlled measurement for the one path this paper
173 concerns. Section 5 is an instance with a measured cost attached.

174 *Measurement protocol is the thing at stake..* The methodological core of this
175 paper is not new. Georges et al. (2007) established that a single execution
176 of a managed-runtime benchmark is not a measurement, and that intervals
177 must be computed across executions rather than within one; Kalibera and
178 Jones (2013) show how to spend a fixed budget across levels of repetition,
179 which is the reasoning behind our choice of many short processes over few
180 long ones. Mytkowicz et al. (2009) showed that environmental details unre-
181 lated to the code under study can produce effects larger than the one being
182 measured, without any obvious mistake, and Curtsinger and Berger (2013)
183 that memory layout alone can do so. Barrett et al. (2017) found that virtual
184 machines frequently do not reach a steady state, Traini et al. (2023) that
185 developers misjudge when warmup ends, and Costa et al. (2021) catalogue how
186 microbenchmarks go wrong in practice. Mytkowicz et al. (2010) showed that
187 sampling profilers disagree on which method is hot, which is why Section 4
188 uses the profiler only as corroboration of a result obtained by differencing and
189 relies on ordering rather than on its percentages. Our contribution is not to
190 these methods but is an instance of what they warn about. The attribution
191 this paper overturns has exactly the structure Mytkowicz et al. describe:
192 a second variable — the bundled OpenSSL version — moving unrecorded
193 alongside the one under study, at a granularity coarse enough to hide it
194 (Section 5).

195 *Environment as a confounder..* Laaber et al. (2019) quantify how far cloud en-
196 vironments degrade microbenchmark reliability, and Leitner and Cito (2016)
197 characterise performance variation across public infrastructure providers.
198 Their concern is variance between nominally identical environments. Ours is
199 adjacent and, we think, less examined: a *systematic* difference between envi-
200 ronments that are not identical and are routinely treated as though they were
201 — two container base images differing in a bundled library version. Laaber
202 et al. are also the reason the second-architecture repetition in Section 5.6 is
203 reported as an ordering check rather than as a replication of magnitudes.

204 *Cryptographic interfaces are hard to use correctly..* The defect reported here
205 is not a cryptographic weakness but a consequence of how a cryptographic
206 interface is invoked, which is a well-documented class. Lazar et al. (2014)
207 found that most of the cryptographic vulnerabilities they studied were misuses

208 of libraries by applications rather than bugs in the libraries. Egele et al. (2013)
209 found cryptographic misuse widespread in deployed applications and Georgiev
210 et al. (2012) found certificate validation broken across a wide range of non-
211 browser software; Nadi et al. (2016) attribute much of it to the interfaces
212 themselves rather than to developer inattention, a position Green and Smith
213 (2016) put directly and Indela et al. (2016) respond to with interface proposals;
214 and Acar et al. (2017) show experimentally that API design materially changes
215 whether developers produce correct code. Detection has been automated by
216 specifying correct usage, as CrySL does (Krüger et al., 2018), and by static
217 analysis at scale (Rahaman et al., 2019). Our finding sits within that line
218 with one difference: the interface here admits a use that is *correct*, and costly
219 only for a reason invisible at the call site. A CrySL-style rule could express
220 “pass a key object, not a string”, but the rule sets we know of encode security
221 properties only, so none would report it.

222 *JWT and JOSE libraries..* Security analyses of JWT implementations consis-
223 tently find key-handling defects. Detering et al. (2017) demonstrated attacks,
224 key confusion among them, against several popular JOSE libraries; Xu et al.
225 (2023) automated the detection of cryptographic vulnerabilities in applica-
226 tions that use JWT; Yang et al. (2026) evaluated 43 JWT libraries across
227 16 programming languages for vulnerabilities; and Shatnawi et al. (2024)
228 surveyed Python JWT libraries for security warnings and adoption. RFC
229 8725 (Sheffer et al., 2020) codifies the resulting practice. All of this work
230 treats a library’s handling of key material as a security surface. Section 7
231 treats the same surface as a cost site, across seven libraries, and Section 8.4
232 shows the two views are coupled: the control flow that costs time in one
233 library is the control flow that enforces its key-type check.

234 *Mining public repositories..* Section 6.5 draws on code hosted publicly, and
235 the hazards of doing so are catalogued by Kalliamvakou et al. (2014) — among
236 them that many repositories are not software projects, that activity is not
237 uniformly distributed, and that a search result is not a random sample. Our
238 sampling frame is stated in Section 3.9 with those hazards in view, which is
239 why no population proportion is computed from it.

240 *The cost of exceptions..* That exceptions are expensive in managed runtimes,
241 and that their cost is concentrated in construction rather than in unwinding,
242 is long established: Ogasawara et al. (2006) optimise around exception paths
243 in a Java virtual machine, and Renwick et al. (2019) quantify the cost in a

244 compiled setting. Parravicini and Mueller (2021) provide the closest available
 245 cost-attribution baseline for the V8 engine, though on speculation rather than
 246 exceptions. Empirical studies of how exceptions are *used* find handlers that
 247 discard what they catch to be common: Cabral and Marques (2007) report
 248 that handlers in the Java and .NET code they studied rarely attempt recovery,
 249 Kery et al. (2016) find that handlers mostly log, print, return or rethrow, and
 250 de Pádua and Shang (2017) measure the prevalence of handling anti-patterns,
 251 as Weimer and Necula (2004) did earlier for error-handling mistakes. That
 252 literature treats a swallowed exception as a correctness and maintainability
 253 smell. The handler at the centre of this paper is a swallowed exception used
 254 as control flow on a hot path, and it is a performance defect rather than
 255 a correctness one: the fallback it guards is correct. None of these studies
 256 measures such a handler’s cost; Section 4.2 does, and finds the exception itself
 257 the smaller part.

258 *Key-type confusion..* That a token declaring an HMAC algorithm must not be
 259 validated against asymmetric key material is long established; RFC 8725 (Shef-
 260 fer et al., 2020) describes the attack and its mitigations. The library studied
 261 here has been hardened against this class twice. Versions before 4.2.2 accepted
 262 an HMAC token verified with an RSA or EC public key used as the shared
 263 secret (CVE-2015-9235) (GitHub Advisory Database, 2015), and versions
 264 up to 8.5.1 could still be led to verify forged tokens from RSA to HMAC
 265 through their key-retrieval path (CVE-2022-23541), alongside two related
 266 defects in key-type and default-algorithm handling (CVE-2022-23539, CVE-
 267 2022-23540) (Auth0, 2022); all three were fixed in version 9.0.0. Section 8
 268 reports no new attack. It reports that the second hardening introduced the
 269 control flow whose cost this paper measures, that the defence depends on
 270 that control flow, and that removing the cost naively removes the defence.

271 2.1. What we searched for and did not find

272 Three areas bear directly on this result and we were unable to locate
 273 verifiable peer-reviewed work in any of them. We state each as an absence
 274 we searched for rather than as a claim about the whole literature; each is
 275 also, in part, why the misattribution corrected here survived as long as it
 276 did. Two adjacent questions — the cost of error construction in V8, and
 277 where key-parsing cost lies inside OpenSSL — were equally unstudied, and
 278 this paper measures both directly (Sections 4.2 and 5.5).

279 *OpenSSL 3.x performance regressions.* We found no peer-reviewed study
280 of performance regressions across the OpenSSL 3.x series. Evidence exists
281 as a maintainer statement (Gaynor and Kehrer, 2026), as project issues —
282 for instance `openssl/openssl` issues #17627 and #20698, which concern
283 performance of the 3.x provider and decoder machinery — and as mailing-list
284 discussion. None was produced under a measurement protocol we can assess,
285 so Section 5.5 measures the path directly rather than adopting the mechanism
286 those sources propose.

287 *Base image to bundled library to application performance.* We found
288 no academic study tracing that chain. Malka et al. (2026) address build
289 reproducibility, Zerouali et al. (2019) package staleness and Cito et al. (2017)
290 ecosystem practice; none measures how the library version an image pins
291 changes application performance.

292 *Cryptographic API misuse as a performance defect.* The misuse literature
293 cited above treats misuse exclusively as a security problem. We found no
294 work treating a correct-but-costly use of a cryptographic interface as a latency
295 or throughput defect, which is precisely what this paper reports.

296 3. Methodology

297 3.1. Unit of measurement

298 The unit is one operating-system process, not one call. Each process
299 performs 40000 timed iterations of a single condition after a warmup of half
300 that number, and contributes its median. Reported figures are medians of
301 per-process medians, with percentile bootstrap intervals taken over processes.

302 An interval computed over the calls inside one process describes that
303 process’s steady state and is far too narrow: it cannot observe the compilation
304 decisions, inline-cache states and heap layouts that differ between runs of the
305 same code (Georges et al., 2007). Conditions in the decomposition of Section 4
306 were run in 15 separate processes of 40000 calls; conditions in the runtime
307 matrix of Section 5 in 10 processes of 20000 calls. The largest between-process
308 coefficient of variation is 3.5% on the host; in the matrix it is 20.2%, occurring
309 on the `probe_succeeds` condition under `node:26.6.0-bookworm` rather than
310 across the matrix generally.

311 Bootstrap intervals over 15 or 10 units are coarse, and we report them
312 as indicative rather than as calibrated coverage. Where an interval collapses
313 to a point — the timer overhead row of Table 1 — it should be read as the

314 clock’s resolution showing through, not as a precise estimate. The per-process
315 values underlying every interval are in the replication package.

316 Conditions are interleaved within each round rather than run in blocks.
317 Under block ordering a thermal excursion or a background process lands
318 entirely on whichever condition is executing and is indistinguishable from an
319 effect; interleaved, it is distributed across conditions and appears as between-
320 round variance. The clock’s own cost is measured in the same process as the
321 condition it accompanies and is reported rather than subtracted: 0.042 μ s
322 against the smallest quantity reported, 0.583 μ s.

323 Differences between conditions are reported throughout as the median
324 of per-round *paired* differences, not as the difference of two medians. The
325 two are not identical and the distinction matters at the magnitudes involved;
326 Section 4.3 and Section 8.4 each rely on it.

327 3.2. *Decomposing the call*

328 `jwt.verify()` is decomposed into the operations it performs, each timed
329 in isolation under the same protocol: structural decode; the HMAC primitive
330 with a string key and with a pre-parsed key; the symmetric key constructor;
331 and the asymmetric key parse the library attempts first, in both the case
332 where it throws and the case where it succeeds.

333 The library measured throughout is `jsonwebtoken` version 9.0.3, cloned
334 at tag v9.0.3. The code path at issue is shown in Figure 1.

335 3.3. *Separating the runtime from the deployment*

336 A host microbenchmark compared against an in-container measurement
337 cannot support an attribution to deployment: those two environments differ
338 in more than deployment, and any of the differences could carry the effect.
339 Here containerisation is held constant and the runtime is varied instead: four
340 container images spanning two OpenSSL series and three V8 major versions,
341 on one machine under one container engine.

342 The pair carrying most of the inference is Node 18 and Node 20: they ship
343 V8 10 and V8 11 respectively while sharing the OpenSSL 3.0 series. In stock
344 images Node and OpenSSL versions otherwise co-vary, which would leave the
345 attribution underdetermined. Because that pairing carries the attribution,
346 the runtime identity of each image — Node, OpenSSL and V8 version, libc,
347 architecture and image digest — is recorded as a datum and cross-checked
348 against the versions each measurement row carried. That check found 0
349 mismatches across the matrix. Section 5.2 states what this design alone

```

120 if (secretOrPublicKey != null && !(secretOrPublicKey instanceof
    KeyObject)) {
121   try {
122     secretOrPublicKey = createPublicKey(secretOrPublicKey);
123   } catch (_) {
124     try {
125       secretOrPublicKey = createSecretKey(typeof
        secretOrPublicKey === 'string' ? Buffer.from(
        secretOrPublicKey) : secretOrPublicKey);
126     } catch (_) {
127       return done(new JsonWebTokenError('secretOrPublicKey is not
        valid key material'))
128     }
129   }
130 }

```

Figure 1: Key resolution in jsonwebtoken version 9.0.3, verify.js, lines 120--130. Verbatim but for the common leading indentation, which is removed to fit the column; line numbers are the file's own. A secret that is not already a `KeyObject` is offered to `createPublicKey()` first, and `createSecretKey()` is reached only from the exception handler. For an HMAC secret supplied as a string the outer `try` can never succeed, so the asymmetric parse is attempted and discarded on every validation.

cannot separate, and Sections 3.4 and 3.5 describe the two further designs that do.

3.4. Taking the JavaScript engine out

Runtime pairs can separate engine from library only as far as stock builds allow, so the attribution was tested by removing the engine altogether. We read the Node source at the two release lines measured (`src/crypto/crypto_keys.cc` at v18.20.8; `deps/ncrypto` at v22.23.3, which issues the same sequence) and reproduced in C, call for call, what `createPublicKey()` asks of OpenSSL when handed an HMAC secret: three `PEM_bytes_read_bio` scans for a public key, an RSA public key and a certificate, each under an error-queue mark and each followed by `BIO_reset`; then, since none was recognised, `PEM_read_bio_PrivateKey` with Node's passphrase callback; then the C side of Node's error path, which drains the OpenSSL error queue into strings. The V8 side of that error path is deliberately absent; it is what Section 3.5 measures.

The program was compiled once per OpenSSL release with one compiler (GCC 13.3.0) and one set of flags, against 8 releases from 3.0.16 to 3.6.1, each built from source with identical configuration and linked by run path so that no system library could be picked up. Each binary reports the version it actually loaded; rows whose loaded version differed from the one requested would be discarded, and there were 0. The protocol is that of Section 3.1: one condition per process, 15 processes per release and condition, 5000 timed calls each, timed with `clock_gettime`, conditions and releases interleaved within rounds. Besides the full sequence we time its parts separately — the three public-key scans, the private-key parse, the error-string drain — and a *successful* parse of an RSA-2048 public key through the same path.

3.5. Separating the exception from the parse

Inside Node the failed parse is an OpenSSL failure followed by a V8 exception: Node constructs an `Error`, attaches `library`, `reason` and `code` properties, throws it through the `createPublicKey()` frame, and the caller discards it. To measure that exception without the parse, a JavaScript benchmark builds the same object — message, property values and error code are captured from a real failure on the runtime being measured, not typed — and throws it from 8 frames below the catch. Variants separate its parts: a plain `Error` thrown and caught in one frame; the Node-shaped error at depth; the same with stack capture disabled; and a Node-internal argument

error raised before any OpenSSL call. The real probe is timed alongside, with and without stack capture, so that the stack’s share of it is obtained by paired differencing. All conditions run under official Node release binaries, which bundle their own OpenSSL, on six runtimes — 18.20.8, 20.20.2, 22.23.3, 23.11.1, 24.21.0 and 26.10.0 — at 15 processes of 10000 calls per condition.

Combining this with Section 3.4 gives a three-way decomposition of the probe on each runtime: the C cost of the OpenSSL release that runtime bundles, the V8 exception, and a remainder we attribute to the binding between them — argument preparation, key-handle allocation and the C++ side of error construction. The three come from different processes and cannot be paired, so the interval on the remainder is obtained by resampling each run’s processes independently.

3.6. Other libraries

To ask whether the defect is one library’s or a design choice’s, we read the key-handling code of six further libraries, chosen as the most used in each ecosystem: `fast-jwt` and `jose` (JavaScript), `PyJWT` (Python), `golang-jwt` (Go), and `nimbus-jose-jwt` and `jjwt` (Java). For each we answer one question from source: given a string HMAC secret, does it attempt to parse it as an asymmetric key? We then time HS256 verification in each with the secret in two forms, under the same one-condition-per-process protocol at 15 processes of 20000 calls. “String” is the form each library’s documentation shows for an HMAC secret, used the way its interface is meant to be used; “pre-parsed” is the library’s own key object, built once. Where a library does not accept a string we use its rawest accepted form and say so. The JavaScript libraries are run under two runtimes, one bundling OpenSSL 3.0 and one bundling 3.5, because the `jsonwebtoken` penalty depends on that. The penalty is the median of per-round paired differences between the two forms.

3.7. A second architecture

Tables 1 and 3 were measured on one ARM64 host. Both were repeated with unchanged scripts on an x86.64 machine (Intel(R) Xeon(R) Processor @ 2.80GHz, four virtual CPUs), including the four container images of Table 3; Sections 3.4–3.6 were measured on the same machine. That machine is a virtual machine on shared cloud infrastructure, not the dedicated host a replication of magnitudes would require (Laaber et al., 2019; Leitner and Cito, 2016). We use it for what it can support: whether the *ordering* of conditions

422 and runtimes holds on a second instruction set, and ratios between conditions
423 measured interleaved in the same rounds. We do not compare its absolute
424 magnitudes with the ARM64 host's.

425 3.8. *In-situ measurement*

426 The deployed service exposes a histogram timing the verification call alone.
427 The instrument is a *paired-scrape difference*: the cumulative histogram is read
428 before and after each arrival-rate step, and the two readings are differenced,
429 which isolates that step's distribution from the cumulative counters. Means
430 are computed as the difference of the histogram's `_sum` divided by the difference
431 of its `_count`, so they are exact and do not depend on bucket boundaries;
432 bucket resolution bounds the reported percentiles only.

433 Seven arrival rates are driven, and the whole sweep is run twice on one
434 process: once in ascending order and once in descending order. We report
435 the *range across the two passes*. This is a range of two, not a confidence
436 interval; no standard error is computed from it, and it is not written as a
437 symmetric tolerance. The opposite-order design also serves as a drift test,
438 which Section 6.2 reports.

439 One process and two passes is the whole of the in-situ evidence. It is
440 sufficient to establish that the cost is present in deployment and of what order;
441 it is not sufficient to characterise the shape of the cost-versus-rate relation,
442 and Section 6.1 is written accordingly.

443 3.9. *Prevalence: three instruments, three frames*

444 Prevalence is approached with three instruments that fail in different ways.
445 Their sampling frames differ and are not interchangeable, so no population
446 proportion is computed from them. All three were captured on 2026-09-18.

447 *Population co-occurrence counts*. Code-search counts over public repositories,
448 in four cells: two languages by two import forms. The denominator of a
449 cell is files importing the library; the numerator is files that *also mention* the
450 pre-parsing interface. This measures co-occurrence of two strings in one file,
451 not verified usage. The denominators are returned rounded — each is an exact
452 multiple of 1024 — so they are magnitudes rather than counts established
453 file by file.

454 *A classified sample of call sites*. Call sites located by search, deduplicated
455 to one file per repository so that no project dominates, with the key argument
456 traced within its file. This is a convenience sample from a relevance-ranked,

undocumented ranking; inclusion probabilities are unknown and no weighting back to the population is possible.

A targeted counter-search. Every file co-occurring the library with the pre-parsing interface, fetched in full and adjudicated by hand. This frame is deliberately biased toward finding the pattern — that is its purpose as a negative control — and it is therefore disqualified as a prevalence estimator.

Three reasons preclude a population-level prevalence. The population numerator and the verified numerator measure different things: of 13 automatically flagged call sites, 5 were false positives on hand inspection, so the co-occurrence signal carries an error rate that would be inherited, unmeasured, by any extrapolation. The sample has no inclusion probabilities. And the counter-search is selected on the outcome. Results are therefore reported *in our sample*, or *among repositories that co-occur the interface and call the validator*, and never as a population proportion. This constraint is carried into the abstract and the conclusion.

3.10. Host state is recorded per measurement

A service measured on a host it shares with its own load generator, ingress and sidecars is measured together with whatever else that host is doing. We observed this directly while building the instrument: re-running one configuration with a background indexing process consuming host CPU moved the point at which throughput collapsed by hundreds of requests per second, and both runs are committed in the replication package (Katha et al., 2026). A curve that moves with unrelated host activity does not characterise the service, and this paper therefore makes no capacity claim. Every in-situ window instead carries the host load average observed during it, committed beside the measurement, and Section 6.3 uses it.

4. RQ1: Where the cost goes

4.1. The decomposition

Table 1 gives each operation in the validation path, timed in isolation on the host under the protocol of Section 3, and Figure 2 plots it.

The two emphasised rows are the result. The signature primitive costs 2.208 μs of a 29.92 μs call. The conversion to which the remainder is most readily attributed costs 0.583 μs . The discarded attempt that precedes that conversion costs 18.04 μs , which is 60% of the whole call and its largest single component.

Table 1: Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes, which with 15 units are indicative rather than calibrated (Section 3.1).

Operation	Median (μs)	95% interval (μs)
<code>verify()</code> , string secret	29.92	[29.75, 30.12]
discarded asym. parse	18.04	[17.83, 18.25]
successful asym. parse	13.83	[13.62, 13.92]
<code>verify()</code> , pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

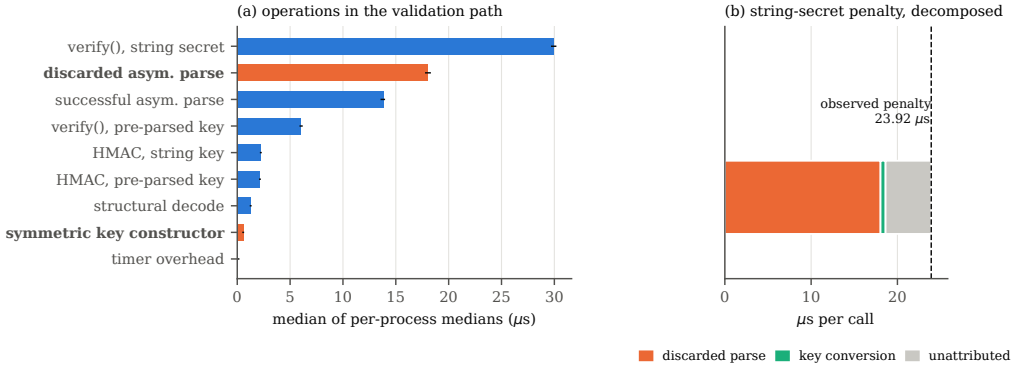


Figure 2: (a) Every operation in Table 1, on a linear scale, with the two components the paper turns on emphasised; whiskers are the 95% percentile bootstrap intervals over 15 processes. (b) The 23.92 μs penalty for supplying a string secret rather than a pre-parsed key, decomposed. The three segments are medians of per-round paired differences, not a partition of a single measurement, and they do not sum to the penalty: the 5.292 μs remainder is unattributed (Section 4.3).

492 Supplying a pre-parsed key instead of a string reduces the call by 23.92 μs
493 [23.58, 24.00] over 15 paired rounds.

494 *The natural repair is also wrong..* Once the signature primitive is shown to be
495 small, one explanation for the remainder suggests itself immediately: that the
496 cost lies in converting key material from a string into a key object on every
497 call. It is the natural explanation, and Table 1 refutes it. That conversion

Table 2: The discarded attempt decomposed, per runtime, in μs (x86_64). *Probe*: `createPublicKey(secret)` inside Node, thrown and caught. *C parse*: the same OpenSSL calls issued from C against the OpenSSL release that runtime bundles, built from source. *V8 exception*: an **Error** of the same shape, message and properties thrown from 8 frames and discarded, with no OpenSSL call. *Binding*: the probe less the other two; its interval, from independent resampling of the three runs, is in the replication package. Medians of per-process medians over 15 processes (probe, exception) and 15 processes (C). The C column uses a stock build of each release, not Node’s own build of it, so the binding column is approximate (Section 10).

Runtime	OpenSSL	Probe	C parse	V8 exception	Binding
Node 18.20.8	3.0.16	390.5	394.4	4.781	-8.694
Node 20.20.2	3.0.19	443.4	431.3	4.796	7.292
Node 23.11.1	3.0.16	383.5	394.4	3.959	-14.88
Node 22.23.3	3.5.8	18.98	8.447	4.021	6.515
Node 24.21.0	3.5.8	19.54	8.447	4.136	6.952
Node 26.10.0	3.5.8	18.01	8.447	3.421	6.145

costs $0.583 \mu\text{s}$, so the attribution is wrong by roughly the ratio between that figure and $18.04 \mu\text{s}$.

A second observation is usually offered in support of the conversion account: an asymmetry between HMAC and RSA costs, read as conversion being dearer than parsing a PEM. It is backwards. A successful asymmetric parse costs $13.83 \mu\text{s}$ against $0.583 \mu\text{s}$ for the conversion. The real asymmetry is that the RSA path’s parse *succeeds*, making its cost useful work, while the HMAC path’s fails, making it waste.

4.2. What the $18.04 \mu\text{s}$ is made of

The $18.04 \mu\text{s}$ is the cost of the discarded attempt taken whole. It contains at least two things: the work OpenSSL performs before concluding that the input is not an asymmetric key, and the construction and unwinding of the exception that returns that conclusion to the library. Table 1 already bounds the split: a *successful* asymmetric parse — the same entry point, reaching a conclusion and throwing nothing — costs $13.83 \mu\text{s}$, the same order as the failing one, so the exception cannot be most of it. Table 2 replaces the bound with a decomposition, combining the C measurement of the same OpenSSL release (Section 3.4) with the exception baseline (Section 3.5) on six official runtimes.

517 Three things follow. First, on every runtime bundling OpenSSL 3.0 the C
 518 parse alone matches the probe to within a few percent — 101% of it under
 519 Node 18.20.8 and 103% under Node 23.11.1, the excess over 100% being the
 520 difference between our build of the release and Node’s. There the discarded
 521 attempt *is* the OpenSSL parse, and it would cost the same with no JavaScript
 522 engine present. Second, the exception is small and nearly constant across
 523 runtimes: the Node-shaped error costs 4.781 μ s under Node 18.20.8 and
 524 3.421 μ s under Node 26.10.0 (1.2% and 19.0% of the respective probes), and
 525 roughly half of it is stack capture (with capture disabled, 2.204 and 1.930 μ s).
 526 Disabling stack capture for the real probe saves 5.397 μ s and 1.767 μ s on
 527 the same two runtimes, the same order as the synthetic figure. Third, on
 528 runtimes bundling OpenSSL 3.5, where the parse is cheap, the three parts
 529 are of comparable size: under Node 26.10.0 the parse is 47% of the probe,
 530 the exception 19.0% and the binding 34.1%.

531 This is why the component is named a discarded *parse attempt* rather
 532 than a discarded exception. Where it is expensive the exception is about a
 533 hundredth of it, and even where it is cheap the exception is a fifth. These
 534 runtimes are Linux builds on x86_64; the ARM64 host figure of Table 1 was
 535 not itself decomposed this way, and its 18.04 μ s is of the same order as the
 536 OpenSSL 3.5 rows here, not the OpenSSL 3.0 rows.

537 4.3. *The parts do not account for all of it*

538 If the penalty for supplying a string were exactly the discarded attempt
 539 plus the conversion, the two would sum to it. They do not, quite:

	observed penalty	23.92 μ s
540	discarded parse + conversion	18.62 μ s
	residual	5.292 μ s

541 All three quantities are medians of per-round paired differences, per
 542 Section 3.1. The residual’s interval, [4.916, 5.501], excludes zero. The two
 543 measured parts therefore account for 77.9% and not more; the remainder
 544 lies in the library’s own type and claim checking along the non-pre-parsed
 545 path. We report it rather than absorbing it into the headline, because a
 546 decomposition whose parts are declared to sum exactly to the whole has
 547 usually not been checked.

548 The pre-parsed path carries a smaller unexplained remainder of the same
 549 kind, and we record it for consistency rather than leaving the asymmetry

unremarked: `verify()` with a pre-parsed key costs $6.042\ \mu\text{s}$ against $1.292\ \mu\text{s}$ of structural decode plus $2.166\ \mu\text{s}$ of HMAC, leaving roughly $2.58\ \mu\text{s}$ unattributed. Neither residual was measured directly; both are the subject of conditions we did not run, and both are stated as unattributed rather than assigned.

4.4. Confirmation without differencing

The decomposition reaches its conclusion by subtracting conditions — the same inference style this paper criticises elsewhere. It is therefore checked against V8’s sampling profiler, which attributes self time to named frames with no arithmetic on our part. At 50000 iterations per run and 3 runs per environment, matched so the percentages are comparable, the asymmetric key parse accounts for a median 66.2% of self time on the host (range 65.3–67.4 across runs), 95.3% (95.1–95.6) under Node 18, and 61.5% (58.5–61.6) under Node 26. The frame does not appear among the top frames of the pre-parsed path in any environment.

Two methods, one answer. We rely on the ordering and the dominance rather than on the exact percentages: sampling attribution depends on rate and on what else the process is doing.

5. RQ2: What governs the magnitude

Table 3 varies the runtime with containerisation held constant, and Figure 3 plots the two columns that matter.

Table 3: The same measurement across four container runtimes on one machine, 10 processes each of 20000 calls. The *Disc. parse*: the discarded asymmetric parse; *Pre-parsed*: `verify()` with a pre-parsed key. [‡]The host row is from Table 1 and is not containerised. [†]Inferred from the run timestamp, not recorded per row: that run predates the per-measurement OpenSSL field (Section 10).

Runtime	OpenSSL	V8	Disc. parse (μs)	Pre-parsed (μs)
Node 18 musl	3.0.16	10	373.79	7.58
Node 20 musl	3.0.19	11	420.62	7.25
Node 26 musl	3.5.7	14	22.94	6.56
Node 26 glibc	3.5.7	14	16.58	6.04
host [‡]	3.6.3 [†]	14	18.04	6.042

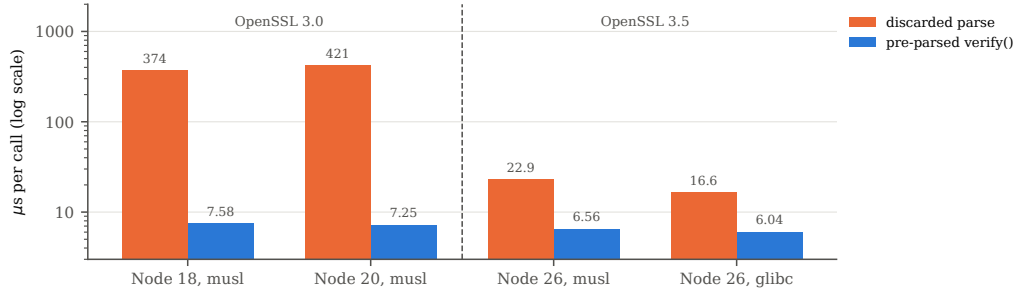


Figure 3: The four container rows of Table 3 on a logarithmic axis. The discarded parse spans $25.36\times$ across images while the pre-parsed call spans $1.26\times$. The dashed rule marks the OpenSSL series boundary. Within this matrix it is not a controlled boundary, because the two fast images also carry a newer engine and a different build (Section 5.2); Section 5.5 controls it. The uncontainerised host row is omitted because its comparison with the container rows is confounded.

5.1. Engine major version does not distinguish the two slow runtimes

Node 18 and Node 20 ship V8 10 and 11 — different major versions of the engine — and share the OpenSSL 3.0 series. Both pay of order 373.79–420.62 μs . Node 26 ships OpenSSL 3.5 and pays 22.94 μs on the same libc. Within the OpenSSL 3.0 series, then, changing the engine’s major version does not move the quantity; had Node 20 been fast, the attribution would have been to the engine.

5.2. What this design cannot separate

That argument establishes a negative within one OpenSSL series. It does not establish the positive attribution for the change between series, and we state the limitation rather than leaving it implicit.

The fast rows are Node 26. They carry OpenSSL 3.5 *and* V8 14 *and* every other change in that build. OpenSSL version is therefore perfectly confounded with engine version and with build configuration across the 3.0–3.5 boundary. Nothing in this matrix can separate them. The absence of variation between V8 10 and V8 11 does not license the inference that V8 14 is also irrelevant; that is a different comparison, and we did not make it.

What the matrix supports on its own is the following, and no more: the magnitude co-varies with the bundled OpenSSL version across four images; the two slow images differ in engine major version and do not differ in magnitude; and the two libc variants of one image differ by less than the

591 difference between series. Separating OpenSSL from the engine requires a
592 crossed design — a runtime pairing a newer engine with an older OpenSSL,
593 or the parse measured outside Node entirely. Section 5.5 reports both, and it
594 is on them, not on this matrix, that the causal wording used in this paper
595 rests.

596 5.3. *The effect is confined to key parsing*

597 Across the four container rows, pre-parsed validation spans $1.26\times$, the
598 HMAC primitive $1.39\times$ and structural decode $1.35\times$. The discarded parse
599 spans $25.36\times$. The successful parse moves with it ($11.74\times$), so this is the cost
600 of key parsing in general rather than anything peculiar to the failure path.
601 The failure is simply the case in which an application pays that cost on every
602 request for a result it discards.

603 5.4. *What this design does not isolate*

604 A reader may notice that the uncontainerised host row is of the same
605 order as the Node 26 container row. That comparison is confounded and we
606 draw no conclusion about containerisation from it: the host runs a different
607 operating system from the containers, so operating system, C library and Node
608 build all change alongside the container boundary. Isolating containerisation
609 would require a Linux-native baseline, which we did not obtain. The claims
610 above concern only the four container rows, among which containerisation is
611 constant.

612 5.5. *Caused by OpenSSL: the parse without an engine*

613 Table 4 removes Node entirely: the OpenSSL calls `createPublicKey()`
614 makes for an HMAC secret, issued from C against 8 OpenSSL releases built
615 identically, and Figure 4 plots its first and last columns.

616 In plain C the failure costs $394.4\ \mu\text{s}$ on OpenSSL 3.0.16 and $8.447\ \mu\text{s}$ on
617 3.5.8, a ratio of 46.69 [46.16, 46.90]. Nothing else differs between those two
618 measurements: the program, the compiler, the flags, the machine and the
619 protocol are the same. The magnitude is therefore *caused* by the OpenSSL
620 release. That is a claim about this code path in these releases; it says nothing
621 about OpenSSL performance generally.

622 The table also says where in OpenSSL the cost lives and when it went
623 away. Almost all of it is the private-key parse — 99% of the full sequence on
624 3.0.16 — the step at which OpenSSL 3 hands the input to its provider-based
625 decoders; the three public-key scans reject the input at the PEM header and

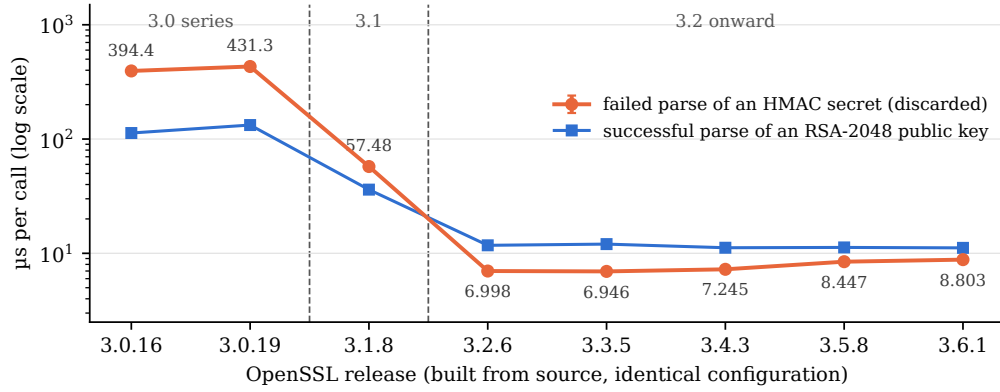


Figure 4: The failed parse of an HMAC secret issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one configuration (Table 4, column *Full*), beside a successful parse of an RSA-2048 public key through the same path. Whiskers are 95% percentile bootstrap intervals over 15 processes and are narrower than the markers. The cost falls in two steps, at 3.1 and 3.2, and is flat thereafter.

Table 4: The failed asymmetric parse issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one compiler and one set of flags (x86_64), in μ s. Medians of per-process medians over 15 processes of 5000 calls. *Full*: everything `createPublicKey()` asks of OpenSSL for an HMAC secret, including the error-string drain; 95% intervals for it are in the replication package and are within 0.9% of the median for every release. Its parts follow. *Successful parse*: an RSA-2048 public key through the same path, for comparison.

OpenSSL	Full	Private-key parse	Public-key scans	Error strings	Successful parse
3.0.16	394.4	390.9	2.287	0.796	112.9
3.0.19	431.3	424.0	2.231	0.869	132.6
3.1.8	57.48	54.84	2.196	0.497	36.09
3.2.6	6.998	4.487	2.057	0.459	11.75
3.3.5	6.946	4.465	2.074	0.474	12.04
3.4.3	7.245	4.554	2.098	0.500	11.19
3.5.8	8.447	5.757	2.051	0.483	11.26
3.6.1	8.803	5.985	2.234	0.472	11.16

are cheap in every release, and the error strings cost under a microsecond.
The fall came in two steps: to 57.48 μ s in 3.1.8, and to 6.998 μ s in 3.2.6, after

Table 5: The runtime matrix of Table 3 on two architectures, same images and scripts, in μs . ARM64: 10 processes of 20000 calls. x86_64, a shared cloud virtual machine: 10 processes of 20000 calls. Absolute values are not comparable across the two machines.

Image	OpenSSL	Discarded parse		Pre-parsed <code>verify()</code>	
		ARM64	x86_64	ARM64	x86_64
<code>node:18.20.8-alpine</code>	3.0.16	373.79	607.1	7.58	12.66
<code>node:20-alpine</code>	3.0.19	420.62	556.3	7.25	11.89
<code>node:26.6.0-alpine</code>	3.5.7	22.94	42.13	6.56	10.94
<code>node:26.6.0-bookworm</code>	3.5.7	16.58	19.91	6.04	9.701

628 which it is flat to within a few microseconds through 3.6.1. We measured the
629 steps, not the source changes that produced them; identifying those would
630 take a bisection of the OpenSSL history that we have not done. A successful
631 parse of real key material fell with it, which is why the effect is confined to
632 key parsing in general rather than to the failure path.

633 *Engine and library crossed..* The official runtimes of Section 3.5 contain the
634 pairing stock container images lack: Node 22.23.3 ships V8 12.4.254.21 with
635 OpenSSL 3.5.8, while Node 23.11.1 ships the *newer* V8 12.9.202.28 with the
636 *older* OpenSSL 3.0.16. The probe costs 18.98 μs on the first and 383.5 μs
637 on the second. The Node 18/20 pair of Table 3 showed that a change of
638 engine at a fixed OpenSSL series leaves the cost where it was; this pair shows
639 that a newer engine does not rescue an older OpenSSL. Together with the C
640 measurement, the engine is excluded from both directions.

641 5.6. A second architecture

642 Table 5 repeats the runtime matrix, with the same images and scripts,
643 on x86_64 (Section 3.7). Absolute values are not compared across the two
644 machines; the question is whether the ordering holds.

645 The property the attribution rests on — every OpenSSL 3.0 image slower
646 than every OpenSSL 3.5 image — holds on both architectures. The strict
647 rank order does not: the two OpenSSL 3.0 images, Node 18 and Node 20,
648 exchange places between ARM64 and x86_64. Their difference is small against
649 the gap between series on either machine, and no claim in this paper depends
650 on their relative order. On x86_64 the discarded parse spans $30.50\times$ across the
651 images while pre-parsed validation spans $1.31\times$, against $25.36\times$ and $1.26\times$ on
652 ARM64. The host decomposition of Table 1 was repeated too (Node 22.22.2,

653 OpenSSL 3.5.5): the discarded parse is 19.42 μ s of a 36.48 μ s call (53%), the
654 symmetric constructor 1.261 μ s and the HMAC primitive 3.105 μ s, so the
655 component ordering of Table 1 holds there as well. The full x86_64 host table
656 is in the replication package.

657 6. RQ3: Deployment and prevalence

658 6.1. In-situ cost across arrival rates

659 The deployed service was driven at seven arrival rates, each measured
660 by the paired-scrape difference of Section 3.8, with the whole sweep run
661 ascending and then descending on one process. Table 6 reports the range
662 across the two passes at each rate for the configuration the service deploys, a
663 string secret, and Figure 5 plots it.

Table 6: In-situ validation cost with a string secret, by arrival rate. The two columns are the ascending and the descending pass over the same seven rates on one process; each is a single window, and the pair is two observations rather than an interval. Host load average during each window is committed with the measurement.

Arrival rate (requests/s)	Mean per call (μ s)	
	ascending	descending
5	928.24	1027.97
10	976.28	1017.34
25	918.54	934.47
50	758.57	737.12
100	571.59	607.78
200	494.91	532.50
400	452.82	467.62

664 One observation follows, and it is the one this section claims: the cost
665 is present in deployment and is far larger than the isolated host figure of
666 Table 1, consistent with Section 5 in that the deployed runtime is the one
667 carrying the older OpenSSL.

668 Two things visible in Table 6 need care, and we set them out explicitly.

669 *Wall-clock exceeds the prediction; processor time does not..* The deployed
670 runtime’s discarded parse costs 373.79 μ s (Table 3), and the remainder of the
671 validation path is of order 7.58 μ s, giving an expected total near 381 μ s. The

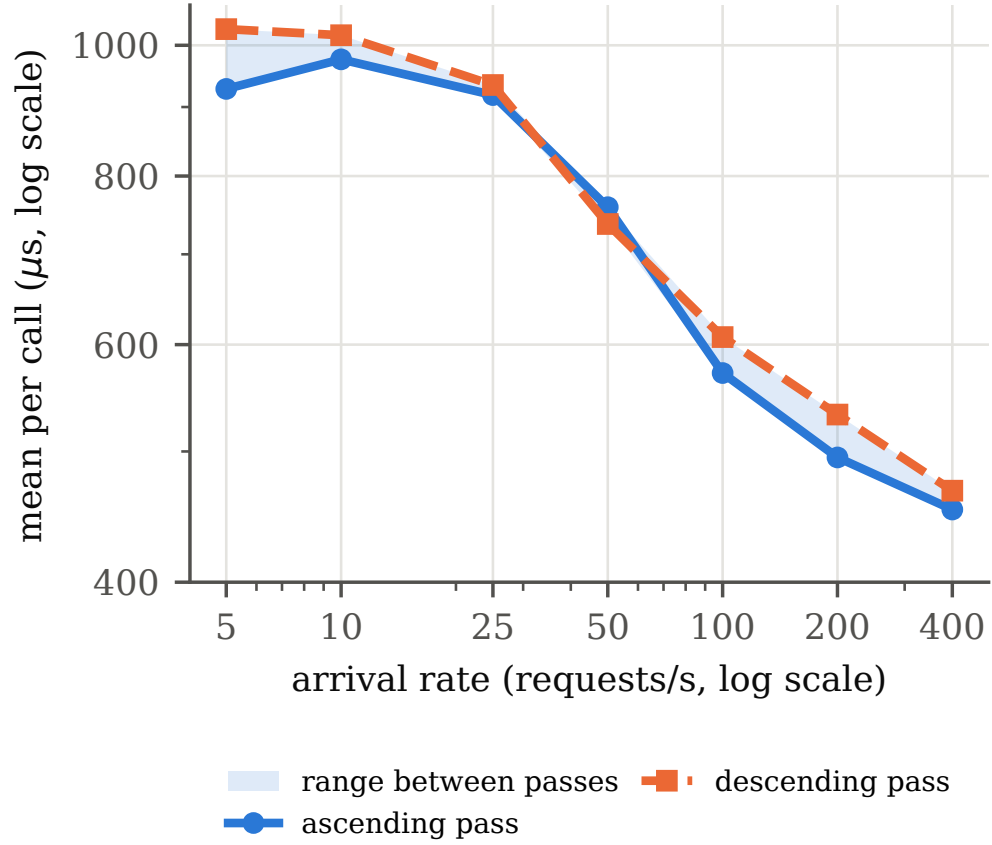


Figure 5: In-situ validation cost against arrival rate, string secret. The two passes are drawn separately rather than averaged: Section 6.2 reports that they differ systematically, and averaging would conceal that. The shaded band is the range between the passes, which is two observations rather than an interval.

672 wall-clock means observed across the sweep are 453–1028 μs , exceeding that
 673 by 71–647 μs . Contention inflates a duration rather than deflating it, so these
 674 are upper bounds, and Section 6.4 shows where the excess is not: measured
 675 as processor time, the validation call costs within 0.8% of the prediction. The
 676 excess is therefore time the call spent off the processor on a shared host, not
 677 additional work. We did not decompose that off-processor time further, and
 678 no claim below depends on it.

679 *The decline with arrival rate is not interpreted..* Cost falls monotonically from
 680 928.24 μs at 5 requests/s to 452.82 μs at 400 requests/s. This is the opposite
 681 of what contention alone would produce, and several mechanisms unrelated
 682 to validation would produce it — a fixed per-window overhead amortising
 683 over more calls, processor frequency scaling and idle-state exit latency at low
 684 rates, cache and branch-predictor residency, or continued tier-up in the engine.
 685 We did not run the conditions that would distinguish them. We therefore
 686 report the numbers and do not claim that validation cost depends on arrival
 687 rate; establishing that would require varying window duration at fixed rate,
 688 pinning the frequency governor, and extending warmup at low rates, none of
 689 which we did.

690 6.2. *The two passes differ systematically*

691 Sweeping up and then down is not only replication; it detects warm-up,
 692 thermal and hysteresis effects. If the difference between passes were noise,
 693 its sign would alternate across rate points. It does not. In the string-secret
 694 sweep the descending pass is slower at 6 of 7 rate points; in the pre-parsed
 695 sweep the descending pass is *faster* at 7 of 7. A consistent sign across seven
 696 points is not what noise produces.

697 This has a direct consequence for how the ranges in Table 6 should be read.
 698 A systematic component means the two passes are not independent samples of
 699 a common value, so the range between them understates uncertainty further
 700 than a range of two already does. We therefore report ranges and draw no
 701 inference that depends on their width. It is also a methodological finding in
 702 its own right: a single-direction sweep would have produced a curve carrying
 703 a consistent bias with no way to detect it.

704 6.3. *Host load, not the condition, drives the outliers*

705 Agreement between passes varies considerably across windows. We dis-
 706 tinguish *relative* disagreement, the gap between passes as a fraction of their

707 mean, from *absolute* disagreement, the gap in microseconds; the two behave
708 differently and the distinction carries this subsection.

709 The widest relative disagreement in the dataset and the tightest occur at
710 the same arrival rate in different sweeps. The wide one is the highest-load
711 window in its own sweep on the descending pass, though not on the ascending
712 pass, whose maximum load falls at a different rate. The tight one, at the
713 same arrival rate in another sweep, sits at a lower load on both passes and
714 its two passes agree to within $0.02\ \mu\text{s}$. Relative disagreement therefore does
715 not track the condition under measurement.

716 We considered two alternative explanations for the relative asymmetry and
717 reject both. A small-mean effect would require absolute disagreement to be
718 roughly constant, so that its relative size grew only as means shrank; absolute
719 disagreement instead varies over three orders of magnitude and is largest
720 where the means are largest, which is the opposite pattern. A measurement
721 floor would require the affected values to sit at the instrument’s resolution;
722 they do not, and in any case the means reported here are computed from the
723 histogram’s sum and count rather than from its buckets, so bucket resolution
724 cannot bound them.

725 The operative consequence is that in-situ figures are reported with the
726 host load recorded during their window, and comparisons between windows
727 at different loads are not treated as comparisons between conditions.

728 6.4. *Corroboration: the cost is processor time, and it matches the prediction*

729 An in-situ figure of this magnitude has two readings. Either it is work the
730 service performs, or it is time the request spends waiting — queued behind
731 other requests, descheduled by a contended host, or inflated by the instrument.
732 We separate them with an A/B of the same service at one arrival rate, with
733 validation enabled in one window and disabled in the next, on the same
734 pod running the same image, and read the processor time the application
735 container was charged from its cgroup accounting rather than from a clock.
736 Table 7 reports it.

737 Enabling validation adds $384.6\ \mu\text{s}$ of processor time per call. The runtime
738 matrix predicts $381\ \mu\text{s}$ for the same work on the same runtime (Section 6.1);
739 the two agree to within 0.8%. They are obtained by instruments that share
740 no code: cgroup CPU accounting on a deployed pod, and a single-process
741 microbenchmark in an isolated container. A request that queues or is de-
742 scheduled accrues wall-clock time but no CPU, so this figure cannot contain
743 queueing delay.

Table 7: Validation enabled against validation disabled, at one arrival rate, two 180 s windows on the same pod and image, 36001 requests each. A signed token is on the wire in both arms; the disabled arm removes only the handler’s `jwt.verify()` call. Application and sidecar CPU are means of one-second cgroup samples in the window; CPU per call is that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99. Host load is a single sample at window start.

	disabled	enabled
Achieved rate (req/s)	200.006	200.006
Application CPU (millicores)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (millicores)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean latency (ms)	1.6802	2.0332
Host load, 1 min	3.53	5.11

744 Three observations weigh against reading the difference as an artefact of
745 the two windows. The sidecar, in the same pod under the same contention,
746 did not rise with the application: 53.40 millicores disabled against 49.13
747 enabled. The one-second CPU samples of the two arms do not overlap at
748 all — disabled never exceeded 100.577 millicores in 127 samples, enabled
749 never fell below 114.963 in 127 — which separates the arms without a
750 distributional assumption. And event-loop lag is unchanged, so neither arm
751 was near saturation. The two windows are two observations, not a sample:
752 the difference carries no interval and we compute none.

753 6.5. How common is the avoiding pattern, in our sample?

754 Three instruments, three frames, reported separately as Section 3.9 re-
755 quires. All counts were captured on 2026-09-18. None of the figures below is
756 a population proportion.

757 *Population co-occurrence..* Of 790528 public JavaScript files importing the
758 library through `require`, 29 also mention the pre-parsing interface; for
759 JavaScript through `import` the figures are 524288 and 38; for TypeScript
760 through `require`, 8288 and 0; and for TypeScript through `import`, 333824
761 and 101. Cells are not pooled into a rate: a file using both import forms
762 would be counted twice, so their sum 1656928 is an upper bound on the union
763 and the largest cell 790528 a lower bound. No cell exceeds 0.0303% of files
764 mentioning the interface. These are search-engine estimates of file counts over
765 public repositories only, they measure co-occurrence rather than use, and the

766 denominators are returned *rounded* — each is an exact multiple of 1024 — so
767 they should be read as magnitudes rather than as counts established file by
768 file.

769 *Classified call sites..* Of 327 call sites classified across 247 distinct repositories
770 in our sample, 0 pass a pre-parsed key. Environment variables account for
771 176 of those call sites, untraced identifiers and member expressions 93, string
772 literals 52, file contents 4 and buffers 2. No file in this corpus contains the
773 pre-parsing interface at all, in any spelling, so the zero reflects the pattern's
774 absence from the sample rather than a classifier that cannot see it. Incidentally,
775 309 of 327 call sites (94%) pass no algorithm restriction, which Section 8.4
776 returns to.

777 Matching only the conventional `jwt.verify()` spelling would filter the
778 corpus toward one import idiom, so call sites are found by resolving the
779 local binding of the import — covering arbitrary aliases, namespace imports,
780 destructured and renamed imports, and direct calls on the require expres-
781 sion. That binding resolution contributes 12 of the call sites above that the
782 conventional spelling misses, of which 0 pass a pre-parsed key.

783 *Targeted counter-search..* A sample can only show a pattern is rare if it
784 could have found it. Every file co-occurring the library with the pre-parsing
785 interface was therefore fetched: 191 files, of which 120 contain a validation
786 call, spanning 112 distinct repositories. An automated pass flagged 13 call
787 sites as passing a pre-parsed key and all were then read by hand: 5 are false
788 positives — projects defining their own helper function of the same name,
789 one of which returns a string — 1 could not be resolved within its file, and 7
790 are genuine, across 6 distinct projects.

791 *The rate, with its denominator..* Among the 112 repositories that both refer-
792 ence the pre-parsing interface and call the validator, 6 use it: 5.36%. That
793 denominator is the most favourable available, since those projects demonstra-
794 bly know the interface exists, and it is not the population. Within this frame
795 the pattern is rare rather than absent, and it is rare even among projects
796 already aware of it. We do not extrapolate beyond the frame.

797 **7. RQ4: One library, or a design choice?**

798 A cost found in one library is an observation about that library. It becomes
799 a statement about a class of defect only if the mechanism can be named

Table 8: HS256 verification across seven JWT libraries in four languages (x86_64), in μ s. *Asym. first?*: given a string HMAC secret, does the library attempt to parse it as an asymmetric key, as read from its source. *String* and *pre-parsed*: the secret in the form the library’s documentation shows, and in the library’s own key object built once (Section 3.6). *Penalty*: median of per-round paired differences, with a 95% bootstrap interval over rounds. Medians of per-process medians over 15 processes of 20000 calls. Absolute costs measure different amounts of work in different runtimes and are not a ranking of libraries. ^aNo string accepted for an HMAC key; the rawest accepted form (bytes) is used. ^bA string is accepted only by a deprecated setter that resolves it once, when the parser is built.

Library	Lang.	Version	Crypto backend	Asym. first?	String	Pre-parsed	Penalty	95% interval
jsonwebtoken	JS	9.0.3	OpenSSL 3.0.19	yes	476.2	10.23	465.9	[465.2, 467.8]
jsonwebtoken	JS	9.0.3	OpenSSL 3.5.8	yes	34.22	9.844	24.56	[24.07, 24.82]
fast-jwt	JS	6.3.3	OpenSSL 3.0.19	no	7.564	7.570	0.070	[-0.116, 0.111]
fast-jwt	JS	6.3.3	OpenSSL 3.5.8	no	7.917	7.934	-0.068	[-0.134, 0.197]
jose	JS	6.2.12	OpenSSL 3.0.19	no ^a	121.2	78.42	42.66	[38.50, 45.89]
jose	JS	6.2.12	OpenSSL 3.5.8	no ^a	98.89	73.98	24.80	[21.91, 26.51]
PyJWT	Python	2.15.0	hashlib, OpenSSL 3.0.13	no	41.33	41.28	-0.103	[-0.457, 0.338]
golang-jwt	Go	5.3.1	Go crypto/hmac	no ^a	4.810	4.764	0.034	[-0.042, 0.117]
nimbus-jose-jwt	Java	10.10	JCA SunJCE	no	4.358	4.229	0.304	[-0.098, 1.016]
jjwt	Java	0.13.0	JCA SunJCE	no ^b	12.15	12.33	-0.406	[-0.715, 0.648]

800 independently of the library and predicts which other libraries have it. The
801 mechanism here can be named: *resolving key material of unknown type by*
802 *attempting the asymmetric parse first, and treating its failure as the signal*
803 *that the material is a shared secret*. Table 8 tests the prediction against six
804 further libraries.

805 *What the source says..* Only **jsonwebtoken** attempts the asymmetric parse.
806 The others resolve the ambiguity in one of three ways, none of which involves
807 a parse that can fail on a shared secret. *Inspect, then dispatch*: **fast-jwt**
808 examines the text of the key — whether it carries a PEM header or is a JSON
809 Web Key — and calls exactly one constructor, skipping even that inspection
810 when every permitted algorithm is HMAC; **PyJWT** likewise inspects the text,
811 and rejects PEM- or SSH-looking material offered as an HMAC key rather
812 than trying to parse it. *Refuse the ambiguous input*: **jose** does not accept
813 a string for an HMAC key, and **golang-jwt** accepts only a byte slice for
814 HMAC verification, returning a key-type error for anything else. *Type the*
815 *key*: **jjwt** requires a **SecretKey** for HMAC and a **PublicKey** for signatures,
816 so the distinction is made by the Java type system; **nimbus-jose-jwt** offers
817 a constructor that takes a string and converts its bytes without parsing them.
818 Refusing a string does not by itself remove per-call work, however: **jose**,
819 handed raw bytes, imports them into a **WebCrypto** key on every call.

820 *What the measurements say..* Of the seven libraries, two pay for being handed
821 their raw form rather than a key object, in the sense that the paired interval
822 on the penalty excludes zero; for the other five it does not, and their point
823 estimates are within a microsecond of zero. The two are `jsonwebtoken`
824 and `jose`, and they pay for different reasons. `jsonwebtoken` performs the
825 failing asymmetric parse: its penalty is 465.9 μ s under OpenSSL 3.0.19 and
826 24.56 μ s under OpenSSL 3.5.8, a string-to-pre-parsed ratio of 46.55 and 3.48
827 respectively. `jose` parses nothing that can fail, but imports the raw bytes
828 into a WebCrypto key on every call: 42.66 μ s and 24.80 μ s under the same
829 two runtimes. Both penalties grow on OpenSSL 3.0, so the OpenSSL release
830 sets the price of per-call key handling generally; but the failing parse is the
831 costliest form of it, about 11 times `jose`'s on OpenSSL 3.0. Two of the
832 libraries that pay nothing resolve the key once, when a verifier or parser is
833 built (`fast-jwt`, `jjwt`), so a raw secret costs nothing per call by construction;
834 that is itself the design point.

835 *What this establishes..* The libraries without the problem matter as much as
836 the ones with it. All seven validate the same kind of token with the same
837 primitive — against the same OpenSSL, in the JavaScript cases — and five pay
838 nothing for a raw secret. The cost therefore follows a design choice, resolving
839 key material on every call, and not JSON Web Token validation, HMAC or
840 OpenSSL as such: OpenSSL 3.0 sets the price, but only a library that makes
841 that choice is charged it, and one that makes it by way of a failing parse is
842 charged the most. The failing parse is exactly what the order-of-evaluations
843 profiler of Selakovic et al. (2017) is designed to find, and the alternatives
844 above are the fixes that line of work suggests: reorder the test, make it cheap,
845 or move it off the per-call path. The comparison does not estimate how many
846 libraries make the choice; it shows that making it is what matters.

847 **8. RQ5: The cost as a security property**

848 Sections 4–7 treat the discarded attempt as a cost. This section asks
849 who can cause it, why it is there, and what removing it does to the library's
850 defences. Its evidence is the library's source at the releases named, the
851 library's own test suite, and the measurements already reported; it introduces
852 no new timing condition, and where a statement rests on reading code rather
853 than on measurement we say so.

854 *8.1. Threat model*

855 The adversary is a remote client that can place arbitrary bytes where the
856 service expects a bearer token, including every field of the token header, and
857 knows which library the service uses. It does not know the shared secret
858 and cannot produce a valid HMAC. The service verifies HS256 tokens with
859 a shared secret supplied in the form the library’s documentation shows, a
860 string — the form every call site in our sample uses (Section 6.5). The asset
861 of interest is key-type separation: a token declaring an HMAC algorithm must
862 never be verified against asymmetric key material, nor the reverse (Sheffer
863 et al., 2020). Out of scope are compromise of the secret, defects elsewhere
864 in the library, and availability. We did not drive adversarial traffic at the
865 deployed service and make no availability or capacity claim; the question here
866 is which inputs reach the discarded attempt, not how many a service can
867 absorb.

868 *8.2. The attempt precedes authentication*

869 Figure 6 shows the order in which `verify()` performs its checks at
870 tag v9.0.3. Reaching the key-resolution block of Figure 1 requires only
871 that the token have three dot-separated segments, a header and payload that
872 decode as JSON, and a non-empty signature segment (lines 59–118). Nothing
873 on that path depends on the secret. The permitted-algorithm list is consulted
874 afterwards, at line 144; the key-type checks at lines 148–160; the signature at
875 line 165.

876 Two consequences follow. A token that will be rejected pays the discarded
877 attempt in full: 18.04 μ s on the host, and 373.79 μ s on the runtime the studied
878 service deploys (Table 3). And restricting permitted algorithms, the defence
879 RFC 8725 recommends and which 94% of the call sites in our sample omit,
880 does not avoid it, because the restriction is enforced after resolution. The cost
881 is therefore under the control of any client that can reach the endpoint, not
882 only of the service’s legitimate load. That is a statement about reachability,
883 established from the source; its magnitude is the one measured in Sections 4–6.

884 *8.3. The attempt arrived with a key-confusion fix*

885 We compared `verify.js` in the published packages of versions 8.5.1, 9.0.0
886 and 9.0.3. In 8.5.1 the secret is passed to the signature routine as given, with
887 no key-object resolution. The resolution block of Figure 1 first appears in
888 9.0.0 and is unchanged, apart from whitespace, through 9.0.3. Version 9.0.0
889 is the release that fixed CVE-2022-23541 and CVE-2022-23539 (Auth0, 2022),

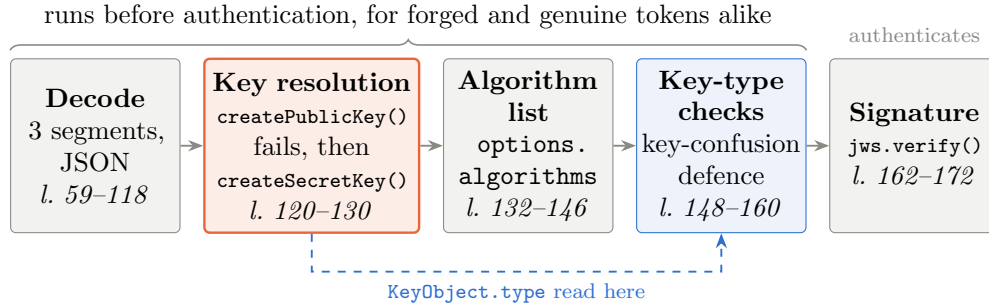


Figure 6: Order of operations in `verify()` at tag v9.0.3, with the file’s line numbers. Every step up to and including the algorithm and key-type checks runs before the signature is verified, and so runs for a forged token exactly as for a genuine one. The discarded attempt sits inside key resolution; the key-type checks that implement the library’s key-confusion defence read the type of the key object resolution produces (dashed arrow). Pinning permitted algorithms is enforced at the third step and therefore does not prevent the second.

890 and the checks implementing those fixes — the comparison of the declared
891 algorithm with the key’s type at lines 148–152 and the validation of asymmetric
892 key types at lines 154–160 — operate on the key object the resolution block
893 produces. The attempt is thus not an oversight in otherwise untouched code.
894 It is part of a security fix, and its price is set by whichever OpenSSL release
895 a deployment happens to bundle (Section 5).

896 This is consistent with the one earlier report of the symptom: HS256
897 verification in the 9.x series measured 34 times slower than in 8.5.1, on a
898 Node release bundling OpenSSL 3.0 (Section 8.6). We did not time version
899 8.5.1 ourselves, and state the connection as consistent with the report rather
900 than as established by it.

901 8.4. A fix, and a constraint on it

902 The fix is to dispatch on the token’s declared algorithm before attempting
903 the asymmetric parse, so that an HMAC token with an ordinary string secret
904 is resolved directly. Applied, this takes the call from 29.92 μ s to 7.208 μ s, a
905 factor of 4.15, and to within 1.208 μ s of supplying a pre-parsed key, with no
906 change in calling code. That residual is the median of the per-round paired
907 differences, not the difference of the two medians; the two are not identical
908 and the former is reported throughout.

909 We tested it against the library’s own suite rather than only against
910 assertions of our own. Cloned at tag v9.0.3, the patch applies cleanly and the

911 suite is unchanged: 511 passing and 0 failing unpatched, 511 and 0 patched.

912 These figures are from the host. The practical benefit is largest where the
913 discarded parse is largest — of order 373.79 μ s on the runtime the service
914 actually deploys (Table 3) — and we did not measure the patched library
915 on the container runtimes. The reported factor of 4.15 is therefore a lower
916 bound on the improvement available in the deployed environment, and we
917 state it as a host result rather than as a general one.

918 *The constraint, which is the more important half..* The discarded attempt is
919 not purely wasteful. The *success* of that parse on asymmetric key material
920 is what produces a key object whose type a later check depends on, and
921 that check is what prevents symmetric and asymmetric material from being
922 interchanged (Sheffer et al., 2020). A change that dispatches on the declared
923 algorithm while relaxing which key material may take the fast path alters
924 which inputs reach that check.

925 The fix must therefore be narrow: the fast path is restricted to string
926 secrets that cannot be asymmetric key material, so anything that might
927 be resolves exactly as it does today. Removing that material restriction —
928 the form of the optimisation we argue against — produces 2 failures in the
929 library’s own suite, both in its key-confusion tests (“should not allow HMAC
930 verification with an RSA key in PEM format” and, under “signing with pub
931 key as symmetric”, “should not verify”). All of these outcomes — the suite
932 unchanged under the narrow patch, two key-confusion failures without its
933 guard — reproduce on the x86_64 machine of Section 3.7 under a different
934 Node and OpenSSL release (Katha et al., 2026).

935 *The existing suite already guards the property..* No new regression test is
936 needed: the library’s existing suite constrains the change, and a contributor
937 proposing the unrestricted optimisation would be caught by tests they are
938 required to run. The finding is therefore not a missing test but a hidden
939 coupling: the cost documented in Section 4 cannot be removed by the obvious
940 route, and the reason is a property the maintainers already test for but which
941 is not evident from the code being changed.

942 The independent defence against this class of confusion is for the caller to
943 restrict permitted algorithms (Sheffer et al., 2020), and 94% of the call sites
944 surveyed in Section 6.5 do not.

Table 9: Which tokens reach the discarded key-parse attempt, for a service configured with an HMAC shared secret, as read from the source at tag v9.0.3 and from the patch of Section 8.4. “Well-formed”: three segments, JSON header and payload, non-empty signature. No row requires knowledge of the secret.

Configuration	Tokens that reach the parse
Stock, string secret	every well-formed token, forged or genuine
Narrow fix, plain string secret (no PEM or JWK form)	every well-formed token whose header declares a non-HMAC algorithm; no such token can be valid for this service
Pre-parsed <code>KeyObject</code>	none

945 *8.5. What the narrow fix leaves reachable*

946 The narrow fix selects its fast path on the token’s declared algorithm,
947 and the declared algorithm is chosen by whoever constructs the token. For a
948 token declaring an HMAC algorithm and a plain string secret the parse is
949 skipped. For a token declaring any other algorithm the patch falls through
950 to the original resolution, so `createPublicKey()` is attempted on the shared
951 secret and fails exactly as it does today; the token is then rejected by the
952 algorithm list or the key-type check, but only after the parse. Table 9 sets
953 out which inputs reach the attempt under each configuration. The narrow
954 fix removes the cost from legitimate traffic, not from traffic an adversary
955 constructs. Only a pre-parsed key object removes it for every input, because
956 the resolution block is skipped entirely when the configured key is already a
957 `KeyObject` (line 120).

958 This follows from the code and was not separately timed: we did not
959 measure the patched library on tokens declaring a non-HMAC algorithm,
960 and the per-call cost such a token incurs is that of the discarded attempt
961 measured in Sections 4 and 5. A variant that dispatched on the *configured*
962 algorithms rather than the declared one would close the gap for callers who
963 restrict permitted algorithms to HMAC, since tokens declaring anything else
964 are rejected regardless; it is the choice `fast-jwt` makes (Section 7). We did
965 not implement or test it, it would help only the minority of callers who restrict
966 algorithms, and it would have to keep the same guard on key material.

967 *8.6. Reported upstream*

968 Before reporting, we searched the library’s issue tracker for earlier reports.
969 One exists: an issue opened in April 2024¹ reports that HS256 verification
970 in the 9.x series is 34 times slower than in 8.5.1, on a Node release that
971 bundles OpenSSL 3.0. It diagnoses no cause and, at the time of writing,
972 has no maintainer response. That is the symptom this paper explains,
973 observed independently two years earlier by a user who could not see its
974 source. We reported the mechanism, with a minimal reproduction, per-
975 runtime measurements and the OpenSSL finding, as a separate issue², and a
976 pull request implementing the narrow form of the fix, with its guard, is open
977 against it. At the time of writing neither has received a maintainer response.
978 We state that rather than omit it: the library is maintained at a slow cadence
979 and a response may postdate publication.

980 **9. Implications for practice**

981 *Pre-parse key material once, at startup..* On the host this removes 23.92 μ s
982 per validation [23.58, 24.00]; on the runtime the service actually deploys,
983 where the discarded parse costs 373.79 μ s, it removes far more. It requires
984 no architectural change, no new component and no movement of a trust
985 boundary, and it is the only remedy in this paper that removes the attempt
986 for every input, including tokens an adversary constructs (Table 9). On a
987 current runtime bundling OpenSSL 3.5 the saving is smaller — 24.56 μ s
988 per validation in Table 8 — but it is not negligible beside the 347.3 μ s
989 of processor time the deployed service spends on a request with validation
990 disabled (Table 7). The two figures come from different machines and are
991 compared as orders of magnitude only.

992 *Treat the base image as part of the measurement..* The same code, the same
993 library and the same architecture differ by 25.36 $\times$ on this path across the
994 container images measured, because they pin different OpenSSL versions. A
995 performance figure quoted without the base image it was obtained on is not
996 reproducible, and a regression appearing after a base-image bump may have
997 nothing to do with the application. Section 5.5 shows that on this path the

¹<https://github.com/auth0/node-jsonwebtoken/issues/966>

²<https://github.com/auth0/node-jsonwebtoken/issues/1046>

998 OpenSSL release is the cause; but the practical point would hold even if it
999 were not, since it is enough that the image changes the number.

1000 *For library authors: do not use a failing parse as a type test..* Section 7 found
1001 three designs that avoid the cost — inspect the material and dispatch, refuse
1002 ambiguous input at the interface, or type the key — and all three ship in
1003 maintained libraries today. What they share is that resolution happens once,
1004 or not at all, rather than on every call. The design that pays most is the one
1005 that asks an expensive parser to answer a cheap question on every request.

1006 *Pin the algorithm..* Independently of anything here, callers should restrict
1007 permitted algorithms (Sheffer et al., 2020). Most of the call sites we surveyed
1008 do not, which leaves them relying entirely on library-internal key typing for a
1009 property they could assert directly in one argument. Pinning does not remove
1010 the cost measured here, which is incurred before the restriction is checked
1011 (Section 8.2); it is an independent defence, not a substitute for pre-parsing.

1012 *For security reviewers: review performance changes to verification as security*
1013 *changes..* The cheapest-looking optimisation of this path fails the library’s
1014 key-confusion tests, and the property it breaks is not evident from the lines
1015 being changed (Section 8.4). Conversely, the security fix that introduced the
1016 path carries a cost that varies by more than an order of magnitude with the
1017 bundled OpenSSL release (Sections 5 and 8.3). Changes to token verification
1018 should be run against the library’s key-confusion tests and timed on the oldest
1019 cryptographic library a deployment may bundle.

1020 9.1. Relocation is not the first remedy

1021 A cost believed to be cryptographic invites relocation to a sidecar or
1022 gateway. The cost measured here is not cryptographic and is largely avoidable
1023 in place, so an architectural remedy that also moves a trust boundary should
1024 not be the first thing reached for. We implemented that relocation for the
1025 service studied here; applying a mesh validation policy duplicates verification
1026 rather than moving it, and making it effective requires the application to
1027 trust a proxy-supplied claims header — a known anti-pattern that needs
1028 a filter stripping client-supplied copies. We report that only as a design
1029 note: it carries no measurement, and the configuration, the observation
1030 and the inconclusive automated probe are documented in the replication
1031 package (Katha et al., 2026).

10. Threats to validity

Two architectures, one of them shared.. The primary measurements come from a single ARM64 host. Section 5.6 repeats them on x86_64, and there the split between OpenSSL series and the component ordering of the host decomposition hold. But the x86_64 machine is a virtual machine on shared cloud infrastructure, whose absolute timings are known to be unreliable (Laaber et al., 2019); we therefore take only orderings and ratios between interleaved conditions from it, and no magnitude in this paper should be read as holding on x86_64 hardware in general. A repetition on a dedicated x86_64 host, which the replication package scripts in one command, would turn the ordering check into a replication of magnitudes. The C probe, the exception baseline and the cross-library comparison were measured only on the x86_64 machine. Their conclusions rest on ratios between conditions measured in the same rounds, which is the kind of quantity a shared machine can support, and the effects they report are far larger than the machine’s noise: the median between-process coefficient of variation is 1.5%, 2.9% and 2.8% in the three runs. The largest values (28.0%, 27.7% and 20.2%) occur on sub-microsecond or JIT-dominated conditions that carry no claim, and medians of per-process medians are insensitive to them. The C probe needs no JavaScript runtime and the scripts rebuild every OpenSSL release from source, so repeating it on the ARM64 host or a dedicated x86_64 machine is one command.

The C probe reproduces Node’s calls, not Node’s build.. Section 5.5 issues the OpenSSL calls Node issues, as read from the Node source, against OpenSSL releases we built. Node builds its bundled OpenSSL with its own configuration, so the C figures approximate rather than reproduce the parse cost inside a given runtime, and the binding column of Table 2 inherits that approximation — it is slightly negative on two OpenSSL 3.0 runtimes for that reason. The causal claim does not inherit it: it compares releases under one build configuration with nothing else varying.

The exception baseline is a reconstruction.. Node constructs the error from C++ through V8’s API; our baseline constructs an error of the same shape, message and properties from JavaScript at a comparable stack depth. The two paths share V8’s error construction and stack capture, and the paired stack-capture measurement on the real probe agrees in order of magnitude with the synthetic one, but they are not the same code path. The ARM64

1067 host figure of Table 1 was not itself decomposed; Table 2 decomposes the
1068 same component on six other runtimes.

1069 *The headline host figure has no per-row OpenSSL version..* The host decom-
1070 position of Section 4 predates the per-measurement OpenSSL field. The
1071 version in force is known from the run timestamp and a note recorded at the
1072 time: OpenSSL 3.6.3, which a package upgrade later that day moved to 3.6.4
1073 under the same Node binary. We report it as inferred, marked as such in
1074 Table 3, rather than as recorded. Three measurements that do carry their
1075 version bound how much this matters. The same probe costs 18.01 μ s under
1076 Node 26.10.0 with OpenSSL 3.5.8, 19.42 μ s in the x86_64 host repetition with
1077 OpenSSL 3.5.5, and 16.58 μ s in the Node 26 container with OpenSSL 3.5.7 —
1078 all of the same order as 18.04 μ s, and all from OpenSSL releases after the
1079 step of Table 4. The headline figure is therefore representative of a post-3.2
1080 OpenSSL, which is all the paper uses it for; it would be wrong by more than
1081 an order of magnitude only if the host had run an OpenSSL 3.0 or 3.1 release,
1082 which the record excludes.

1083 *Ranges of two, with a systematic component..* The in-situ dispersion is the
1084 range across an ascending and a descending pass. It is not a confidence
1085 interval, and Section 6.2 shows the two passes differ systematically rather
1086 than randomly, so they are not independent samples of a common value and
1087 the range understates uncertainty further. No inference in this paper depends
1088 on the width of those ranges.

1089 *One in-situ process, two passes..* The rate sweep rests on one process sweeping
1090 seven rates twice. That is sufficient for the claim made — the cost is present
1091 in deployment and of this order — and insufficient for any claim about the
1092 shape of the cost-versus-rate relation, which Section 6.1 accordingly does
1093 not make. The magnitude claim does not rest on the sweep alone: the
1094 A/B of Section 6.4 is a separate pair of windows, measured with a separate
1095 instrument, and it agrees with the microbenchmark prediction on processor
1096 time. The wall-clock excess over that prediction is attributed to time off
1097 the processor and not decomposed further; the decline with arrival rate has
1098 several candidate explanations unrelated to validation that we did not test
1099 (Section 6.1).

1100 *Sampling frames are not interchangeable..* The three prevalence instruments
1101 of Section 3.9 have different units and different selection mechanisms, and none

1102 supports a population proportion. The co-occurrence counts measure two
1103 strings appearing in one file, with a false-positive rate established at 5 of 13 on
1104 hand inspection. The classified sample has unknown inclusion probabilities, a
1105 limitation of repository search noted more generally by Kalliamvakou et al.
1106 (2014). The counter-search is selected on the outcome.

1107 *The matcher’s coverage..* Call sites are found by resolving the local binding
1108 of the import, which covers aliases, namespace imports, destructured and
1109 renamed imports and direct calls on the require expression. It does not resolve
1110 bindings across files, so a project that constructs a pre-parsed key in one file
1111 and validates in another is invisible to this analysis. This is the strongest
1112 threat to Section 6.5; within-file tracing found no such case, which is weak
1113 evidence against it.

1114 *Host load, and the confound that motivated recording it..* A throughput curve
1115 measured on a host shared with its own load generator moves with unrelated
1116 activity on that host, as Section 3.10 describes, which is why no capacity claim
1117 appears in this paper. Every in-situ window here carries the load average
1118 observed during it, and Section 6.3 shows that window-to-window agreement
1119 tracks that load rather than the condition. Contention inflates a duration
1120 rather than deflating it, so in-situ figures are upper bounds; but comparisons
1121 between windows at different loads are not comparisons between conditions,
1122 and we do not treat them as such.

1123 *The A/B is two windows at unequal load..* The two arms of Table 7 are
1124 consecutive windows rather than interleaved rounds, and their host load
1125 is known only at the instant each began. That bars comparing them on
1126 wall-clock time, and we do not. It does not bar the CPU comparison the
1127 section rests on: contention inflates elapsed time, whereas a container’s own
1128 cgroup accounting charges it only the processor time it used. Attaching the
1129 per-second host sampler to a re-run of both arms, interleaved, would remove
1130 the limitation; we did not.

1131 *Token caching is not in the comparison..* A service that caches verified tokens
1132 pays this cost once per token rather than once per request. We did not
1133 measure such a configuration, so the per-request figures here do not describe
1134 a caching deployment.

1135 *One rater..* The 13 automatically flagged call sites were read and classified
1136 by the first author, and the authors are not disinterested: the paper’s claim
1137 is stronger the more of them are false positives, and 5 were classified that
1138 way. There was no second rater and no inter-rater agreement to report. The
1139 per-case verdicts, the evidence for each and the hashes of the files they were
1140 read from are published so the adjudication can be repeated, which is weaker
1141 than having repeated it.

1142 *The security analysis is from source..* Section 8 establishes reachability and
1143 provenance by reading `verify.js` at the releases named and by running the
1144 library’s own tests; it does not drive forged tokens at the deployed service,
1145 time the patched library on tokens declaring a non-HMAC algorithm, or
1146 time version 8.5.1. Its statements are therefore about which inputs reach the
1147 attempt and when the code arrived, not about throughput under attack, and
1148 the paper makes no availability claim.

1149 *Seven libraries, one version each..* Section 7 covers seven libraries in four
1150 languages, chosen as the most used in each ecosystem, each at one release.
1151 Other libraries, and other releases of these, may resolve keys differently; each
1152 classification is from source at the version measured. The comparison supports
1153 the claim that the cost follows a design choice rather than JWT validation; it
1154 does not estimate how many libraries make that choice.

1155 **11. Conclusion**

1156 The per-request cost of token validation in this library is real, is worth
1157 attending to, and is not the signature: the primitive is 2.208 μ s of a 29.92 μ s
1158 call. It is not key conversion either, which costs 0.583 μ s. It is a parse
1159 attempted in the wrong order, whose failure is constructed and thrown away
1160 on every request at 18.04 μ s, 60% of the call. Where that attempt is expensive
1161 it is almost entirely OpenSSL work; the exception that ends it costs a few
1162 microseconds.

1163 It is also a security property of the path. The attempt runs before the
1164 algorithm check and the signature, so any well-formed token can cause it; it
1165 entered the library with a key-confusion fix and that fix depends on it, so
1166 removing it naively fails the library’s key-confusion tests; and the narrow fix
1167 that keeps the defence still leaves it reachable through an attacker-chosen
1168 algorithm header. Pre-parsing the key once, at startup, is both the fastest
1169 configuration and the only one no input can push onto the slow path.

1170 Its price is set by the OpenSSL release a base image happens to pin,
1171 varying $25.36\times$ across runtimes while the rest of the validation path varies
1172 by $1.26\times$ — and set causally, not only by association: the same calls from C
1173 cost $394.4\ \mu\text{s}$ on OpenSSL 3.0.16 and $8.447\ \mu\text{s}$ on 3.5.8, and a newer engine
1174 paired with an older OpenSSL is the slow runtime, not the fast one.

1175 It is a design choice, not a property of JWT: of six other libraries in four
1176 languages none attempts the parse, five pay nothing for a raw secret, and the
1177 one that re-imports its key on every call pays an order of magnitude less. The
1178 symptom had been reported upstream two years before this study without a
1179 diagnosis; we have reported the mechanism.

1180 The pattern that avoids the cost is rare in our sample: among 112
1181 repositories that already reference the pre-parsing interface and call the
1182 validator, 6 use it. No population proportion follows.

1183 Two explanations that present themselves before the measurement are
1184 refuted by it. That the dominant cost is key conversion is refuted by measur-
1185 ing the conversion. That a throughput collapse under layered enforcement
1186 characterises the service is refuted by the collapse point moving with unrelated
1187 host activity, which is why no capacity claim appears here. Both have the
1188 same shape, and it is the shape this paper’s methodology is built to avoid:
1189 a cost attributed to the variable under study while a second variable moves
1190 alongside it, unrecorded at the granularity at which it could change. The
1191 bundled OpenSSL version in Section 5 is the same hazard caught in time —
1192 the quantity varies $25.36\times$ with a component most measurement protocols
1193 record once per run, if at all. The lesson is not that one should measure
1194 in deployment. It is narrower: record the identity of the component whose
1195 behaviour is being measured, on every measurement, at the granularity at
1196 which it can change.

1197 CRediT authorship contribution statement

1198 **Jannatul Ferdous Katha:** Conceptualization, Methodology, Software,
1199 Investigation, Data curation, Formal analysis, Visualization, Writing – original
1200 draft. **Tasmia Tahmid Prova:** Validation, Writing – review & editing. **Md**
1201 **Kishor Morol:** Supervision, Writing – review & editing. **Tze Hui Liew:**
1202 Supervision, Project administration, Writing – review & editing. **Dip Nandi:**
1203 Supervision, Writing – review & editing.

1204 **Declaration of competing interest**

1205 The authors declare that they have no known competing financial interests
1206 or personal relationships that could have appeared to influence the work
1207 reported in this paper.

1208 **Funding**

1209 This research did not receive any specific grant from funding agencies in
1210 the public, commercial, or not-for-profit sectors.

1211 **Data availability**

1212 The data supporting the results of this study are openly available
1213 in the replication package (Katha et al., 2026) at [https://github.com/](https://github.com/Katha-Sikdar/microservices-api)
1214 [Katha-Sikdar/microservices-api](https://github.com/Katha-Sikdar/microservices-api), directory `psao-artifact/`. It contains
1215 all measurements, the instrumentation, the analysis code and the scripts that
1216 generate every numeric value and every figure in this paper, including the
1217 C program and the script that builds each OpenSSL release, the exception
1218 baseline, and the cross-library harnesses in four languages with their pinned
1219 dependency manifests. Every number is generated from a file under the
1220 run or survey directories; none is entered by hand. The survey corpus is
1221 third-party source and is not redistributed; a manifest pinning every file
1222 examined by content hash stands in for it, covering 502 files across 457
1223 repositories. Content hashes resolved for all of them; the repository HEAD
1224 commit resolved for 501 of 502, one repository having become unavailable
1225 between fetch and manifest. The content hash is the authoritative pin and is
1226 complete.

1227 **Ethics and responsible disclosure**

1228 This study measures software behaviour and publicly available source
1229 code; it involves no human participants and no personal data. No attack
1230 was run against any system the authors do not control. The key-confusion
1231 behaviour discussed in Section 8 is an existing, publicly documented attack
1232 class tested only through the library’s own test suite; the upstream report
1233 describes the constraint at the level of mechanism and does not include an
1234 exploitation recipe.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to restructure the manuscript for this journal’s format and to draft revisions of its text. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- Acar, Y., Backes, M., Fahl, S., Garfinkel, S., Kim, D., Mazurek, M.L., Stransky, C., 2017. Comparing the usability of cryptographic APIs, in: 2017 IEEE Symposium on Security and Privacy (SP). doi:10.1109/SP.2017.52.
- Auth0, 2022. CVE-2022-23539, CVE-2022-23541, CVE-2022-23540: Security update for jsonwebtoken. Auth0 security bulletin, 21 December 2022, <https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jwebtoken>. Accessed 2026-09-28.
- Barrett, E., Bolz-Tereick, C.F., Killick, R., Mount, S., Tratt, L., 2017. Virtual machine warmup blows hot and cold. Proceedings of the ACM on Programming Languages 1, 52:1–52:27. doi:10.1145/3133876.
- Cabral, B., Marques, P., 2007. Exception handling: A field study in Java and .NET, in: ECOOP 2007 – Object-Oriented Programming, Springer. pp. 151–175. doi:10.1007/978-3-540-73589-2_8.
- Chen, J., Shang, W., 2017. An exploratory study of performance regression introducing code changes, in: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). doi:10.1109/ICSME.2017.13.
- Cito, J., Schermann, G., Wittern, J.E., Leitner, P., Zumberi, S., Gall, H.C., 2017. An empirical analysis of the Docker container ecosystem on GitHub, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR). doi:10.1109/MSR.2017.67.
- Coarfa, C., Druschel, P., Wallach, D.S., 2006. Performance analysis of TLS web servers. ACM Transactions on Computer Systems 24. doi:10.1145/1124153.1124155.

1267 Costa, D., Bezemer, C.P., Leitner, P., Andrzejak, A., 2021. What’s wrong
1268 with my benchmark results? Studying bad practices in JMH benchmarks.
1269 IEEE Transactions on Software Engineering 47, 1452–1467. doi:10.1109/
1270 TSE.2019.2925345.

1271 Curtsinger, C., Berger, E.D., 2013. STABILIZER: Statistically sound perfor-
1272 mance evaluation, in: Proceedings of the 18th International Conference on
1273 Architectural Support for Programming Languages and Operating Systems
1274 (ASPLOS ’13), pp. 219–228. doi:10.1145/2451116.2451141.

1275 Davis, J.C., Williamson, E.R., Lee, D., 2018. A sense of time for JavaScript
1276 and Node.js: First-class timeouts as a cure for event handler poisoning, in:
1277 Proceedings of the 27th USENIX Security Symposium, pp. 343–359.

1278 Detering, D., Somorovsky, J., Mainka, C., Mladenov, V., Schwenk, J., 2017.
1279 On the (in-)security of JavaScript object signing and encryption, in: Pro-
1280 ceedings of the 1st Reversing and Offensive-oriented Trends Symposium
1281 (ROOTS ’17). doi:10.1145/3150376.3150379.

1282 Egele, M., Brumley, D., Fratantonio, Y., Kruegel, C., 2013. An empirical
1283 study of cryptographic misuse in Android applications, in: Proceedings of
1284 the 2013 ACM SIGSAC Conference on Computer and Communications
1285 Security (CCS ’13). doi:10.1145/2508859.2516693.

1286 Gaynor, A., Kehrer, P., 2026. The state of OpenSSL for pyca/cryptography.
1287 pyca/cryptography project statement, [https://cryptography.io/
1288 en/latest/statements/state-of-openssl/](https://cryptography.io/en/latest/statements/state-of-openssl/). Maintainer statement, not
1289 peer reviewed; accessed 2026-09-24.

1290 Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java
1291 performance evaluation, in: Proceedings of the 22nd Annual ACM SIG-
1292 PLAN Conference on Object-Oriented Programming Systems, Languages
1293 and Applications (OOPSLA ’07). doi:10.1145/1297027.1297033.

1294 Georgiev, M., Iyengar, S., Jana, S., Anubhai, R., Boneh, D., Shmatikov, V.,
1295 2012. The most dangerous code in the world: Validating SSL certificates
1296 in non-browser software, in: Proceedings of the 2012 ACM Conference on
1297 Computer and Communications Security (CCS ’12). doi:10.1145/2382196.
1298 2382204.

1299 GitHub Advisory Database, 2015. Verification bypass in jsonwebtoken (CVE-
1300 2015-9235, GHSA-c7hr-j4mj-j2w6). [https://github.com/advisories/](https://github.com/advisories/GHSA-c7hr-j4mj-j2w6)
1301 [GHSA-c7hr-j4mj-j2w6](https://github.com/advisories/GHSA-c7hr-j4mj-j2w6). Accessed 2026-09-28.

1302 Gong, L., Pradel, M., Sen, K., 2015. JITProf: Pinpointing JIT-unfriendly
1303 JavaScript code, in: Proceedings of the 10th Joint Meeting on Foundations
1304 of Software Engineering (ESEC/FSE '15). doi:10.1145/2786805.2786831.

1305 Green, M., Smith, M., 2016. Developers are not the enemy! the need for usable
1306 security APIs. IEEE Security & Privacy 14. doi:10.1109/MSP.2016.111.

1307 Indela, S., Kulkarni, M., Nayak, K., Dumitras, T., 2016. Helping Johnny
1308 encrypt: Toward semantic interfaces for cryptographic frameworks, in:
1309 Proceedings of the 2016 ACM International Symposium on New Ideas, New
1310 Paradigms, and Reflections on Programming and Software (Onward! 2016).
1311 doi:10.1145/2986012.2986024.

1312 Jin, G., Song, L., Shi, X., Scherpelz, J., Lu, S., 2012. Understanding and
1313 detecting real-world performance bugs, in: Proceedings of the 33rd ACM
1314 SIGPLAN Conference on Programming Language Design and Implementa-
1315 tion (PLDI '12), pp. 77–88. doi:10.1145/2254064.2254075.

1316 Jones, M.B., Bradley, J., Sakimura, N., 2015. JSON Web Token (JWT).
1317 Request for Comments RFC 7519. Internet Engineering Task Force. doi:10.
1318 17487/RFC7519.

1319 Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in:
1320 Proceedings of the 2013 International Symposium on Memory Management
1321 (ISMM '13). doi:10.1145/2464157.2464160.

1322 Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian,
1323 D., 2014. The promises and perils of mining GitHub, in: Proceedings of
1324 the 11th Working Conference on Mining Software Repositories (MSR 2014).
1325 doi:10.1145/2597073.2597074.

1326 Kaltenecker, C., Mühlbauer, S., Grebhahn, A., Siegmund, N., Apel, S., 2023.
1327 Performance evolution of configurable software systems: An empirical study.
1328 Empirical Software Engineering 28. doi:10.1007/s10664-023-10338-3.

1329 Katha, J.F., Prova, T.T., Morol, M.K., Liew, T.H., Nandi, D., 2026.
1330 Replication package: where the cost of JWT verification actually

1331 goes. <https://github.com/Katha-Sikdar/microservices-api>. Direc-
1332 tory `psao-artifact/`.

1333 Kery, M.B., Le Goues, C., Myers, B.A., 2016. Examining programmer
1334 practices for locally handling exceptions, in: Proceedings of the 13th
1335 International Conference on Mining Software Repositories (MSR '16), pp.
1336 484–487. doi:10.1145/2901739.2903497.

1337 Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M., 2018. CrySL: An
1338 extensible approach to validating the correct usage of cryptographic APIs,
1339 in: 32nd European Conference on Object-Oriented Programming (ECOOP
1340 2018), pp. 10:1–10:27. doi:10.4230/LIPIcs.ECOOP.2018.10.

1341 Laaber, C., Leitner, P., 2018. An evaluation of open-source software micro-
1342 benchmark suites for continuous performance assessment, in: Proceedings
1343 of the 15th International Conference on Mining Software Repositories (MSR
1344 '18). doi:10.1145/3196398.3196407.

1345 Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking
1346 in the cloud. how bad is it really? Empirical Software Engineering 24.
1347 doi:10.1007/s10664-019-09681-1.

1348 Lazar, D., Chen, H., Wang, X., Zeldovich, N., 2014. Why does cryptographic
1349 software fail? A case study and open problems, in: Proceedings of the
1350 5th Asia-Pacific Workshop on Systems (APSys '14). doi:10.1145/2637166.
1351 2637237.

1352 Leitner, P., Cito, J., 2016. Patterns in the chaos—a study of performance
1353 variation and predictability in public IaaS clouds. ACM Transactions on
1354 Internet Technology 16. doi:10.1145/2885497.

1355 Liu, Y., Xu, C., Cheung, S.C., 2014. Characterizing and detecting performance
1356 bugs for smartphone applications, in: Proceedings of the 36th International
1357 Conference on Software Engineering (ICSE '14), pp. 1013–1024. doi:10.
1358 1145/2568225.2568229.

1359 Malka, J., Zacchiroli, S., Zimmermann, T., 2026. Docker does not guarantee
1360 reproducibility. arXiv preprint arXiv:2601.12811. No DOI registered;
1361 verified against the arXiv record.

1362 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing
1363 wrong data without doing anything obviously wrong!, in: Proceedings of the
1364 14th International Conference on Architectural Support for Programming
1365 Languages and Operating Systems (ASPLOS '09). doi:10.1145/1508244.
1366 1508275.

1367 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2010. Evaluating
1368 the accuracy of Java profilers, in: Proceedings of the 31st ACM SIGPLAN
1369 Conference on Programming Language Design and Implementation (PLDI
1370 '10), pp. 187–197. doi:10.1145/1806596.1806618.

1371 Nadi, S., Krüger, S., Mezini, M., Bodden, E., 2016. Jumping through hoops, in:
1372 Proceedings of the 38th International Conference on Software Engineering
1373 (ICSE '16). doi:10.1145/2884781.2884790.

1374 Naser, A., Sadeghi Lahijani, M., Wu, C., Gavahi, M., Hoang, V.T., Wang,
1375 Z., Yuan, X., 2020. Performance evaluation and modeling of cryptographic
1376 libraries for MPI communications. arXiv preprint arXiv:2010.06139. No
1377 DOI registered; verified against the arXiv record.

1378 Nistor, A., Jiang, T., Tan, L., 2013a. Discovering, reporting, and fixing
1379 performance bugs, in: Proceedings of the 10th Working Conference on
1380 Mining Software Repositories (MSR '13). doi:10.1109/MSR.2013.6624035.

1381 Nistor, A., Song, L., Marinov, D., Lu, S., 2013b. Toddler: Detecting perfor-
1382 mance problems via similar memory-access patterns, in: Proceedings of
1383 the 35th International Conference on Software Engineering (ICSE '13), pp.
1384 562–571. doi:10.1109/ICSE.2013.6606602.

1385 Ogasawara, T., Komatsu, H., Nakatani, T., 2006. EDO: Exception-directed
1386 optimization in Java. ACM Transactions on Programming Languages and
1387 Systems 28. doi:10.1145/1111596.1111598.

1388 de Pádua, G.B., Shang, W., 2017. Studying the prevalence of exception
1389 handling anti-patterns, in: Proceedings of the 25th International Conference
1390 on Program Comprehension (ICPC '17), pp. 328–331. doi:10.1109/ICPC.
1391 2017.1.

1392 Parravicini, A., Mueller, T., 2021. The cost of speculation: Revisiting over-
1393 heads in the V8 JavaScript engine, in: 2021 IEEE International Symposium

1394 on Workload Characterization (IISWC). doi:10.1109/IISWC53511.2021.
1395 00013.

1396 Rahaman, S., Xiao, Y., Afrose, S., Shaon, F., Tian, K., Frantz, M., Kantar-
1397 cioglu, M., Yao, D.D., 2019. CryptoGuard: High precision detection of
1398 cryptographic vulnerabilities in massive-sized Java projects, in: Proceedings
1399 of the 2019 ACM SIGSAC Conference on Computer and Communications
1400 Security (CCS '19), pp. 2455–2472. doi:10.1145/3319535.3345659.

1401 Renwick, J., Spink, T., Franke, B., 2019. Low-cost deterministic C++ ex-
1402 ceptions for embedded systems, in: Proceedings of the 28th International
1403 Conference on Compiler Construction (CC 2019). doi:10.1145/3302516.
1404 3307346.

1405 Selakovic, M., Glaser, T., Pradel, M., 2017. An actionable performance
1406 profiler for optimizing the order of evaluations, in: Proceedings of the
1407 26th ACM SIGSOFT International Symposium on Software Testing and
1408 Analysis (ISSTA '17). doi:10.1145/3092703.3092716.

1409 Selakovic, M., Pradel, M., 2016. Performance issues and optimizations in
1410 JavaScript: An empirical study, in: Proceedings of the 38th International
1411 Conference on Software Engineering (ICSE '16), pp. 61–72. doi:10.1145/
1412 2884781.2884829.

1413 Shatnawi, A., Al-Duwairi, B., Smarneh, A., 2024. Comprehensive empirical
1414 study of Python JWT libraries. *Procedia Computer Science* 238, 827–832.
1415 doi:10.1016/j.procs.2024.06.099.

1416 Sheffer, Y., Hardt, D., Jones, M.B., 2020. JSON Web Token Best Current
1417 Practices. Request for Comments RFC 8725. Internet Engineering Task
1418 Force. doi:10.17487/RFC8725.

1419 Sikeridis, D., Kampanakis, P., Devetsikiotis, M., 2020. Assessing the overhead
1420 of post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the
1421 16th International Conference on Emerging Networking EXperiments and
1422 Technologies (CoNEXT '20). doi:10.1145/3386367.3431305.

1423 Song, L., Lu, S., 2014. Statistical debugging for real-world performance
1424 problems, in: Proceedings of the 2014 ACM International Conference
1425 on Object Oriented Programming Systems Languages & Applications
1426 (OOPSLA '14), pp. 561–578. doi:10.1145/2660193.2660234.

1427 Staicu, C.A., Pradel, M., 2018. Freezing the web: A study of ReDoS vul-
 1428 nerabilities in JavaScript-based web servers, in: Proceedings of the 27th
 1429 USENIX Security Symposium, pp. 361–376.

1430 Stefan, P., Horký, V., Bulej, L., Tůma, P., 2017. Unit testing performance in
 1431 Java projects: Are we there yet?, in: Proceedings of the 8th ACM/SPEC
 1432 International Conference on Performance Engineering (ICPE '17), pp. 401–
 1433 412. doi:10.1145/3030207.3030226.

1434 Traini, L., Cortellessa, V., Di Pompeo, D., Tucci, M., 2023. Towards effective
 1435 assessment of steady state performance in Java software: Are we there yet?
 1436 Empirical Software Engineering 28. doi:10.1007/s10664-022-10247-x.

1437 Tuveri, N., Brumley, B.B., 2019. Start your ENGINES: Dynamically loadable
 1438 contemporary crypto, in: 2019 IEEE Cybersecurity Development (SecDev).
 1439 doi:10.1109/SecDev.2019.00014.

1440 Weimer, W., Necula, G.C., 2004. Finding and preventing run-time error
 1441 handling mistakes, in: Proceedings of the 19th Annual ACM SIGPLAN
 1442 Conference on Object-Oriented Programming, Systems, Languages, and
 1443 Applications (OOPSLA '04). doi:10.1145/1028976.1029011.

1444 Xu, B., Jia, S., Lin, J., Zheng, F., Ma, Y., Liu, L., Gu, X., Song, L.,
 1445 2023. JWTKey: Automatic cryptographic vulnerability detection in JWT
 1446 applications, in: Computer Security – ESORICS 2023, Springer. doi:10.
 1447 1007/978-3-031-51479-1_14.

1448 Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W.,
 1449 Wang, B., 2026. Token time bomb: Evaluating JWT implementations for
 1450 vulnerability discovery, in: Proceedings of the Network and Distributed
 1451 System Security Symposium (NDSS '26).

1452 Zaman, S., Adams, B., Hassan, A.E., 2012. A qualitative study on performance
 1453 bugs, in: Proceedings of the 9th IEEE Working Conference on Mining
 1454 Software Repositories (MSR '12), pp. 199–208. doi:10.1109/MSR.2012.
 1455 6224281.

1456 Zerouali, A., Mens, T., Robles, G., Gonzalez-Barahona, J.M., 2019. On
 1457 the relation between outdated Docker containers, severity vulnerabilities,
 1458 and bugs, in: Proceedings of the 26th IEEE International Conference on

1459 Software Analysis, Evolution and Reengineering (SANER '19), pp. 491–501.
1460 doi:10.1109/SANER.2019.8668013.

1461 Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X.,
1462 He, P., Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R., 2023.
1463 Dissecting overheads of service mesh sidecars, in: Proceedings of the 2023
1464 ACM Symposium on Cloud Computing (SoCC '23). doi:10.1145/3620678.
1465 3624652.